

순환 부호 (Cyclic Code)

목차

배경 - 순환 부호의 연구 역사

I. 기본 개념 — 순환 부호의 정의와 동기 - 순환 부호란 무엇인가? - 순환 이동 불변성은 왜 구현에 유리한가?

II. 다항식 표현 — 벡터에서 다항식으로 - 순환 부호를 다항식으로 어떻게 표현하는가?

III. 대수적 구조 — 알고 보니 이데알이다 - 순환 부호의 대수적 구조는 무엇인가? - 몫환의 이데알 개수와 생성 다항식의 결정

IV. 생성 다항식과 패리티 검사 다항식 - 생성 다항식이란 무엇이고, 어떤 성질을 가지는가? - 패리티 검사 다항식이란 무엇인가?

V. 행렬 표현과 인코딩 - 생성 행렬은 어떻게 만드는가? - 패리티 검사 행렬은 어떻게 만드는가? - 비체계적 인코딩과 체계적 인코딩의 차이는 무엇인가? - 체계적 인코딩은 구체적으로 어떻게 수행하는가?

VI. 디코딩과 오류 정정 - 신드롬(Syndrome)이란 무엇이고, 오류 검출은 어떻게 하는가? - 순환 부호의 디코딩(복호) 과정은 어떻게 되는가? - 순환 부호의 오류 정정 능력은 어떻게 결정되는가?

VII. 응용과 확장 - 순환 부호가 실무에서 중요한 이유는 무엇인가? - 대표적인 순환 부호에는 어떤 것들이 있는가? - 준순환 부호와 양자내성암호(HQC)는 무엇인가?

부록 - 연습문제 - SageMath 실습 코드

참고문헌 - 참고문헌

순환 부호의 연구 역사

순환 부호는 부호 이론(Coding Theory) 전체의 발전과 밀접하게 얽혀 있다. 아래는 선형 부호 및 순환 부호의 주요 이정표를 시간순으로 정리한 것이다.

1948 — 부호 이론의 탄생

Claude Shannon이 논문 A Mathematical Theory of Communication [1]을 발표하여 정보 이론을 창시하였다. 채널 부호화 정리(Channel Coding Theorem)를 통해 잡음이 있는 채널에서도 오류 확률을 임의로 낮출 수 있는 부호가 존재함을 증명하였으나, 구체적인 부호 구성법은 제시하지 않았다. 이 정리가 이후 수십 년간 “좋은 부호를 어떻게 만들 것인가”라는 연구의 출발점이 되었다.

1950 — Hamming 부호

Richard Hamming이 단일 오류 정정이 가능한 $[2^r - 1, 2^r - 1 - r]_2$ 선형 부호를 제안하였다 [2]. 이것은 최초의 체계적인 오류 정정 부호로, 특정 파라미터에서 순환 부호 구조를 가진다. 본 문서의 $[7, 4]_2$ 부호가 대표적인 예이다.

1954 — Reed-Muller 부호

Irving Reed와 David Muller가 독립적으로 다중 오류 정정이 가능한 Reed-Muller 부호를 발표하였다 [3] [4]. 이 부호는 불 함수(Boolean Function)의 대수적 구조에 기반하며, 1969년 Mariner 9 화성 탐사선의 사진 전송에 실제로 사용되었다.

1957 — 순환 부호의 대수적 기초

Eugene Prange가 순환 부호의 개념을 처음으로 형식화하고, 다항식 몫환 $\mathbb{F}_q[X]/\langle X^n - 1 \rangle$ 의 이데알 구조와의 대응을 확립하였다 [5]. 이 연구가 본 문서의 “순환 부호의 대수적 구조” 및 “몫환의 이데알 개수” 절에서 다루는 대수적 구조의 원형이다.

1959-1960 — BCH 부호

Alexis Hocquenghem(1959) [6]과 Raj Bose, Dwijendra Ray-Chaudhuri(1960) [7]가 독립적으로 BCH(Bose-Chaudhuri-Hocquenghem) 부호를 발견하였다. 갈루아 확대체 위에서 생성 다항식의 근을 지정함으로써 설계 거리(designed distance)를 보장하는 최초의 체계적 방법론이었다. 본 문서의 “순환 부호의 오류 정정 능력” 절에서 다루는 BCH 한계 정리가 이 결과에 해당한다.

1960 — Reed-Solomon 부호

Irving Reed와 Gustave Solomon이 비이진 체 위의 순환 부호인 Reed-Solomon(RS) 부호를 제안하였다 [8]. 심볼 단위 오류 정정에 탁월하여 CD, DVD, QR코드, 심우주 통신 등에 광범위하게 적용되었다.

1961 — Peterson의 디코딩 알고리즘

W. Wesley Peterson이 BCH 부호의 대수적 디코딩 절차를 체계화하고, 저서 Error-Correcting Codes를 출간하였다 [9]. 이 책은 순환 부호 이론의 최초의 종합 교과서로 평가된다.

1968 — Berlekamp-Massey 알고리즘

Elwyn Berlekamp [10]과 James Massey [11]가 오류 위치 다항식을 효율적으로 구하는 반복 알고리즘을 개발하였다. 이 알고리즘은 BCH/RS 부호의 실용적 디코딩을 가능하게 한 핵심 기술이며, 본 문서의 “순환 부호의 디코딩” 절에서 언급한 대수적 디코딩의 근간이다.

1970년대 — CRC의 산업 표준화

순환 부호의 오류 검출 특화 버전인 CRC(Cyclic Redundancy Check)가 이더넷, 통신 프로토콜, 저장 매체 등의 산업 표준으로 채택되었다. LFSR 기반의 초고속 하드웨어 구현이 가능하다는 점이 표준화의 핵심 동력이었다.

1977 — Goppa 부호와 McEliece 암호 시스템

V. D. Goppa가 대수기하학적 부호(Algebraic Geometry Code)의 기초를 놓았고 [12], Robert McEliece가 Goppa 부호를 기반으로 최초의 부호 기반 공개키 암호 시스템을 제안하였다 [13]. 이 시스템은 양자 컴퓨터에도 안전한 것으로 알려져 있으며, 본 문서의 “준순환 부호와 양자내성암호(HQC)” 절에서 다루는 현대 양자내성암호의 이론적 원형이다.

1990년대 이후 — 현대적 발전

1993년 Turbo 부호 [14], 1996년 LDPC 부호의 재발견 [15] 등 반복 디코딩(Iterative Decoding) 기반의 용량 접근 부호(Capacity-Approaching Code)가 등장하면서 부호 이론의 패러다임이 크게 전환되었다. 그러나 순환 부호는 하드웨어 효율성과 대수적 구조의 명확성 덕분에 플래시 메모리, 위성 통신, 양자내성암호 등의 분야에서 여전히 핵심적인 역할을 수행하고 있다.

연구 접근 방향의 전환

순환 부호의 역사에서 주목할 점은, 연구의 접근 방향이 시대에 따라 전환되었다는 것이다.

초기 (1950년대): 공학적 필요 → 대수적 구조의 발견

"순환 이동해도 부호어인 부호를 만들면 LFSR로 구현할 수 있다"

- 순환 이동에 닫힌 선형 부호를 탐색 (Prange, 1957)
- 수학적으로 분석해보니 $F_q[X]/\langle X^n-1 \rangle$ 의 이데알과 동치
- 기존 대수학(이데알 이론)을 그대로 활용 가능

이 시기에는 원하는 공학적 성질(순환 이동 불변성, 하드웨어 효율성)이 먼저 있었고, 그 성질을 엄밀하게 기술하다 보니 이데알 구조와 자연스럽게 연결된 것이다.

전환기 (1959년~): 대수적 구조 → 원하는 성능의 부호 설계

"갈루아 확대체에서 생성 다항식의 근을 지정하면 최소 거리를 보장할 수 있다"

- BCH 부호 (Hocquenghem 1959, Bose-Ray-Chaudhuri 1960)
- Reed-Solomon 부호 (Reed-Solomon 1960)
- 설계 거리 δ 를 미리 지정하여 t-오류 정정 능력을 제어

이 시점부터는 대수적 구조를 적극적으로 활용하여 "원하는 오류 정정 능력을 가진 부호"를 체계적으로 설계하는 방향으로 전환되었다.

현대 (1970년대~): 양방향 상호작용

대수적 구조 $\longleftrightarrow$ 공학적 응용

- CRC: 순환 구조의 하드웨어 효율성 활용 (오류 검출)
- Goppa/McEliece: 부호 이론 → 공개키 암호 시스템
- HQC: 준순환 구조 → 양자내성암호의 키 크기 축소

현대에는 대수적 구조와 공학적 응용이 양방향으로 영향을 주고받으며 발전하고 있다.

순환 부호란 무엇인가?

순환 부호(Cyclic Code)는 선형 블록 부호(Linear Block Code)의 특수한 부분집합으로, **선형성**과 **순환 이동 불변성**이라는 두 가지 핵심 성질을 동시에 만족하는 오류 정정 부호이다.

길이가 n 인 선형 부호 $\mathcal{C}$ 가 다음 두 조건을 모두 만족하면 순환 부호라 한다:

1. **선형성 (Linearity)**: 임의의 두 부호어 $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$ 에 대해 $\mathbf{c}_1 + \mathbf{c}_2 \in \mathcal{C}$
2. **순환 이동 불변성 (Cyclic Shift Property)**: 부호어 $\mathbf{c} = (c_0, c_1, \dots, c_{n-1}) \in \mathcal{C}$ 이면, 오른쪽으로 한 칸 순환 이동시킨 벡터 $\mathbf{c}^{\gg 1} = (c_{n-1}, c_0, c_1, \dots, c_{n-2})$ 도 반드시 $\mathcal{C}$ 에 속한다.

두 칸, 세 칸, ... 임의의 i 칸 순환 이동한 벡터도 모두 유효한 부호어가 된다.

예제 1.1: 순환 이동 확인

$[7, 4]_2$ 이진 순환 부호에서 $\mathbf{c} = (1, 0, 0, 1, 0, 1, 1)$ 이 부호어라면: - 1칸 순환 이동: $\mathbf{c}^{\gg 1} = (1, 1, 0, 0, 1, 0, 1) \rightarrow$ 부호어 - 2칸 순환 이동: $\mathbf{c}^{\gg 2} = (1, 1, 1, 0, 0, 1, 0) \rightarrow$ 부호어 - 3칸 순환 이동: $\mathbf{c}^{\gg 3} = (0, 1, 1, 1, 0, 0, 1) \rightarrow$ 부호어

이 모든 벡터가 부호 공간 $\mathcal{C}$ 에 속해야 한다.

예제 1.2: 순환 부호인지 판별하기

다음 부호가 순환 부호인지 확인해 보자. $n = 4$ 이고 부호어 집합이 다음과 같다고 하자:

$$\mathcal{C} = \{(0, 0, 0, 0), (1, 0, 1, 0), (0, 1, 0, 1), (1, 1, 1, 1)\}$$

- 선형성 검사: $(1, 0, 1, 0) \oplus (0, 1, 0, 1) = (1, 1, 1, 1) \in \mathcal{C}$ OK (다른 조합도 확인 가능)
- 순환 이동 검사:
 - $(1, 0, 1, 0)$ 을 1칸 이동 $\rightarrow (0, 1, 0, 1) \in \mathcal{C}$ OK
 - $(0, 1, 0, 1)$ 을 1칸 이동 $\rightarrow (1, 0, 1, 0) \in \mathcal{C}$ OK

- $(1, 1, 1, 1)$ 을 1칸 이동 $\rightarrow (1, 1, 1, 1) \in \mathcal{C}$ OK

따라서 이 부호는 순환 부호이다.

예제 1.3: 순환 부호가 아닌 경우

$n = 4$ 이고 부호어 집합:

$$\mathcal{C}' = \{(0, 0, 0, 0), (1, 1, 0, 0), (0, 0, 1, 1), (1, 1, 1, 1)\}$$

- $(1, 1, 0, 0)$ 을 1칸 순환 이동 $\rightarrow (0, 1, 1, 0) \rightarrow \mathcal{C}'$ 에 속하지 않음 X

따라서 이 부호는 선형 부호이지만 순환 부호는 아닙니다.

순환 이동 불변성은 왜 구현에 유리한가?

순환 부호가 처음 연구된 동기는 순수 대수학이 아니라 **하드웨어 구현의 효율성**이었다. 1950년대 Prange [5]가 순환 부호를 형식화한 핵심 이유는, 순환 이동에 달려있는 부호가 시프트 레지스터(Shift Register)만으로 인코딩과 오류 검출을 수행할 수 있다는 관찰이었다.

일반 선형 부호의 문제점

일반 $[n, k]_q$ 선형 부호에서 인코딩은 메시지 벡터 $\mathbf{m}$ 과 생성 행렬 G 의 행렬 곱 $\mathbf{c} = \mathbf{m} \cdot G$ 로 수행된다. 이 연산은:

- $k \times n$ 크기의 행렬 G 를 메모리에 저장해야 한다
- $k \cdot n$ 번의 곱셈과 덧셈이 필요하다
- 전체 메시지 블록이 도착할 때까지 기다려야 연산을 시작할 수 있다

예를 들어 $[31, 21]_2$ 부호라면 $21 \times 31 = 651$ 개의 행렬 원소를 저장하고, 매 블록마다 651번의 GF(2) 연산을 수행해야 한다.

순환 부호의 해결책: 시프트 레지스터

순환 부호에서는 모든 부호어가 생성 다항식 $g(X)$ 의 배수이고, 인코딩은 다항식 나눗셈(나머지 계산)으로 귀결된다. 다항식 나눗셈은 LFSR(Linear Feedback Shift Register) 회로로 구현할 수 있는데, 이 회로는:

- $g(X)$ 의 차수 $r = n - k$ 개의 **1비트 레지스터(플립플롭)**만 필요하다
- 데이터 비트가 들어올 때마다 **1클럭에 1비트씩** 실시간 처리한다
- 마지막 비트가 통과하는 순간 레지스터에 남은 값이 곧 패리티(또는 신드롬)이다

$[31, 21]_2$ 부호라면 레지스터 10개와 XOR 게이트 몇 개면 끝이다.

예제: 행렬 곱 vs LFSR — $[7, 4]_2$ 부호 비교

항목	일반 선형 부호 (행렬 곱)	순환 부호 (LFSR)
저장 공간	$G \in GF(2)^{4 \times 7} \rightarrow 28\text{비트}$	$g(X) = 1 + X + X^3 \rightarrow 4\text{비트}$ (계수: $[1, 1, 0, 1]$)
연산량 (1블록)	$4 \times 7 = 28$ 회 AND/XOR	7클럭 (비트당 1회 XOR)
처리 방식	블록 전체 도착 후 일괄 처리	비트 단위 실시간(On-the-fly)
지연(Latency)	블록 길이에 비례	거의 0
회로 복잡도	k -입력 XOR 트리 n 개	r 개 플립플롭 + XOR 게이트

상세 분석: 왜 7클럭이 소요되는가? (비체계적 인코딩 기준)

비체계적 인코딩 $c(X) = m(X)g(X)$ 은 수학적으로 두 다항식의 **컨벌루션(Convolution)** 연산이다. $[7, 4]_2$ 부호에서 메시지는 4비트($k = 4$), 생성 다항식은 차수가 3이므로 4개의 계수($r + 1 = n - k + 1 = 4$)를 가지며, 결과로 나오는 부호어는 7비트($n = 7$)가 된다.

하드웨어(시리얼 곱셈기) 관점에서 비트가 처리되는 과정을 클럭 단위로 분해하면 다음과 같다.

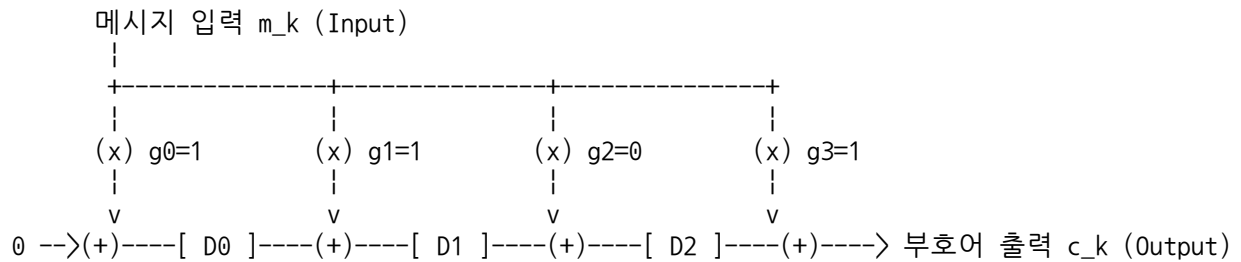
1. 다항식 곱셈의 계수 전개 $m(X) = m_0 + m_1X + m_2X^2 + m_3X^3$, $g(X) = g_0 + g_1X + g_2X^2 + g_3X^3$
(예: $1 + X + X^3$ 이면 $g_2 = 0$)

부호어 $c(X) = \sum_{k=0}^6 c_kX^k$ 의 각 계수 c_k 는 다음과 같이 결정됩니다: $c_0 = m_0g_0$ · $c_1 = m_0g_1 + m_1g_0$
· $c_2 = m_0g_2 + m_1g_1 + m_2g_0$ · $c_3 = m_0g_3 + m_1g_2 + m_2g_1 + m_3g_0$ · $c_4 = m_1g_3 + m_2g_2 + m_3g_1$
· $c_5 = m_2g_3 + m_3g_2$ · $c_6 = m_3g_3$

2. 클럭별 동작 분석 (시리얼 처리) 시리얼 통신 환경에서는 매 클럭마다 메시지 비트가 입력되고, 동시에 부호어 비트가 하나씩 출력됩니다.

클럭	입력 (Input)	출력 (Output)	설명
1	m_0	c_0	m_0 가 들어오자마자 첫 번째 비트 출력
2	m_1	c_1	m_1 과 입력되어 있던 m_0 가 곱셈기 회로를 통과하며 출력
3	m_2	c_2	메시지 입력 완료. 하지만 연산 결과(꼬리 부분)가 아직 다 안 나옴
4	m_3	c_3	
5	0 (Shift)	c_4	피드의 0을 넣어 나머지 연산 결과값들을 밀어냄
6	0 (Shift)	c_5	마지막 비트 출력 완료
7	0 (Shift)	c_6	

3. 하드웨어 구조 (ASCII Art) 시리얼 곱셈기(Serial Multiplier)는 생성 다항식 $g(X)$ 의 계수에 따라 다음과 같이 구성됩니다. 아래는 $g(X) = 1 + X + X^3$ ($g_0 = 1, g_1 = 1, g_2 = 0, g_3 = 1$)의 경우를 도식화한 것입니다.



동작 원리: 1. 클럭 1~4 (**Input Phase**): 메시지 비트 m_0, m_1, m_2, m_3 가 순차적으로 입력됩니다. 매 클럭마다 입력 비트는 각 계수 g_i 와 곱해지고, 지연 소자(D, Flip-flop)에 저장된 이전 값들과 더해져 c_0, c_1, c_2, c_3 가 즉시 출력됩니다. 2. 클럭 5~7 (**Flush Phase**): 모든 메시지 비트가 입력된 후, 입력단을 0으로 고정하고 시프트를 3번 더 수행합니다. 이때 지연 소자(D_0, D_1, D_2)에 남아 있던 중간 연산 값들이 차례로 밀려 나오

면서 나머지 부호어 비트 c_4, c_5, c_6 가 출력됩니다. 3. **결과**: 총 7클럭이 지나면 모든 레지스터가 0으로 비워지고(Flush), 완성된 7비트 부호어 전체가 전송됩니다.

결론: 메시지 비트 수($k = 4$)만큼 데이터를 밀어 넣은 후, 생성 다항식의 차수($n - k = 3$)만큼 추가로 시프트하여 연산을 마무리해야 하므로 총 $4 + 3 = 7$ 클럭이 소요됩니다. 이는 정확히 부호어의 길이 n 과 일치합니다.

4. 시점별 동작 상세 (Step-by-Step Visualization) 메시지 $\mathbf{m} = (1, 1, 0, 1)$ 을 $[7, 4]_2$ 부호($g(X) = 1 + X + X^3$)로 인코딩하는 과정을 한 단계씩 추적합니다. 각 클럭에서의 레지스터 상태와 연산 결과를 확인할 수 있습니다.

[초기 상태] 모든 레지스터(D)는 0으로 시작합니다.

Input m_k: -
Registers: [D0:0] [D1:0] [D2:0]
Output c_k: -

[클럭 1] 첫 번째 정보 비트 $m_0 = 1$ 입력 - $c_0 = m_0g_0 \oplus 0 = 1 \cdot 1 \oplus 0 = 1$ - 레지스터 갱신: $D_0 \leftarrow m_0g_1 \oplus 0 = 1$, $D_1 \leftarrow m_0g_2 \oplus 0 = 0$, $D_2 \leftarrow m_0g_3 = 1$

Input m_k: 1
↓
v (x1) (x1) (x0) (x1) <-- g 계수 곱셈
↓ ↓ ↓ ↓
0 --(+)-- [D0] --(+)-- [D1] --(+)-- [D2] --(+)--> Output c_0: 1
(New:1) (New:0) (New:1)

[클럭 2] 두 번째 정보 비트 $m_1 = 1$ 입력 - $c_1 = m_1g_0 \oplus D_0 = 1 \cdot 1 \oplus 1 = 0$ - 레지스터 갱신: $D_0 \leftarrow m_1g_1 \oplus D_1 = 1 \oplus 0 = 1$, $D_1 \leftarrow m_1g_2 \oplus D_2 = 0 \oplus 1 = 1$, $D_2 \leftarrow m_1g_3 = 1$

Input m_k: 1
↓
v
0 --(+)-- [D0:1] --(+)-- [D1:0] --(+)-- [D2:1] --(+)--> Output c_1: 0
(New:1) (New:1) (New:1)

[클럭 3] 세 번째 정보 비트 $m_2 = 0$ 입력 - $c_2 = m_2g_0 \oplus D_0 = 0 \cdot 1 \oplus 1 = 1$ - 레지스터 갱신: $D_0 \leftarrow m_2g_1 \oplus D_1 = 0 \oplus 1 = 1$, $D_1 \leftarrow m_2g_2 \oplus D_2 = 0 \oplus 1 = 1$, $D_2 \leftarrow m_2g_3 = 0$

Input m_k: 0
↓
v
0 --(+)-- [D0:1] --(+)-- [D1:1] --(+)-- [D2:1] --(+)--> Output c_2: 1
(New:1) (New:1) (New:0)

[클럭 4] 네 번째 정보 비트 $m_3 = 1$ 입력 (입력 완료) - $c_3 = m_3g_0 \oplus D_0 = 1 \cdot 1 \oplus 1 = 0$ - 레지스터 갱신: $D_0 \leftarrow m_3g_1 \oplus D_1 = 1 \oplus 1 = 0$, $D_1 \leftarrow m_3g_2 \oplus D_2 = 0 \oplus 0 = 0$, $D_2 \leftarrow m_3g_3 = 1$

Input m_k: 1
↓
v
0 --(+)-- [D0:1] --(+)-- [D1:1] --(+)-- [D2:0] --(+)--> Output c_3: 0
(New:0) (New:0) (New:1)

[클럭 5] Flush 시작 (입력 = 0) - $c_4 = 0 \cdot g_0 \oplus D_0 = 0$ - 레지스터 갱신: $D_0 \leftarrow 0 \oplus D_1 = 0$, $D_1 \leftarrow 0 \oplus D_2 = 1$, $D_2 \leftarrow 0 = 0$

Input m_k: 0
↓
v

0 --(+)--[D0:0]--(+)--[D1:0]--(+)--[D2:1]--(+)--> Output c_4: 0
 (New:0) (New:1) (New:0)

[클럭 6] Flush 2회차 (입력 = 0) - $c_5 = 0 \cdot g_0 \oplus D_0 = 0$ - 레지스터 갱신: $D_0 \leftarrow 0 \oplus D_1 = 1$, $D_1 \leftarrow 0 \oplus D_2 = 0$, $D_2 \leftarrow 0 = 0$

Input m_k: 0
 |
 v
 0 --(+)--[D0:0]--(+)--[D1:1]--(+)--[D2:0]--(+)--> Output c_5: 0
 (New:1) (New:0) (New:0)

[클럭 7] 마지막 비트 출력 (입력 = 0) - $c_6 = 0 \cdot g_0 \oplus D_0 = 1$ - 레지스터가 모두 비워짐 (Flush 완료)

Input m_k: 0
 |
 v
 0 --(+)--[D0:1]--(+)--[D1:0]--(+)--[D2:0]--(+)--> Output c_6: 1
 (New:0) (New:0) (New:0)

최종 결과: 출력된 비트들을 순서대로 모으면 부호어 벡터 $\mathbf{c} = (1, 0, 1, 0, 0, 0, 1)$ 이 됩니다. 이는 다항식 곱셈 $c(X) = (1 + X + X^3)(1 + X + X^3) = 1 + X^2 + X^6$ 의 결과와 정확히 일치합니다.

왜 “순환”이 시프트 레지스터를 가능하게 하는가

핵심은 순환 이동 = X 곱셈 (mod $X^n - 1$)이라는 대응이다.

시프트 레지스터의 기본 동작은 “모든 비트를 한 칸씩 밀고, 피드백을 넣는 것”이다. 이것은 정확히 다항식에 X 를 곱하고 $g(X)$ 로 나머지를 취하는 연산과 동일하다. 순환 부호가 아닌 일반 선형 부호에서는 이 대응이 성립하지 않으므로, 시프트 레지스터만으로는 인코딩/디코딩을 수행할 수 없다.

정리하면:

순환 이동 불변성

- 부호어 = $g(X)$ 의 배수
- 인코딩 = 다항식 나눗셈
- LFSR로 구현 가능
- 저장 공간 $O(n-k)$, 연산 $O(n)$, 실시간 처리

이것이 1950년대에 순환 부호가 주목받은 이유이며, CRC가 이더넷·USB 등 거의 모든 통신 프로토콜의 오류 검출 표준으로 채택된 근본적인 이유이다.

순환 부호를 다항식으로 어떻게 표현하는가?

길이 n 인 벡터 $\mathbf{c} = (c_0, c_1, \dots, c_{n-1})$ 를 다항식 변수 X 를 사용하여 $(n-1)$ 차 이하의 다항식으로 매핑한다:

$$c(X) = c_0 + c_1X + c_2X^2 + \dots + c_{n-1}X^{n-1}$$

이 표현에서 **오른쪽으로 한 칸 순환 이동**은 수학적으로 다음과 정확히 일치한다:

$$\text{순환 이동} \longleftrightarrow X \cdot c(X) \pmod{X^n - 1}$$

따라서 순환 부호의 모든 연산은 **다항식 몫환(Quotient Ring)** $R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle$ 위에서 이루어진다.

예제 2.1: 벡터 ↔ 다항식 변환

$n = 7$ 일 때: | 벡터 | 다항식 | |——|——-| | $(1, 0, 1, 1, 0, 0, 0)$ | $1 + X^2 + X^3$ | | $(0, 1, 0, 0, 1, 1, 0)$ | $X + X^4 + X^5$ | | $(1, 1, 1, 1, 1, 1, 1)$ | $1 + X + X^2 + X^3 + X^4 + X^5 + X^6$ |

예제 2.2: 순환 이동의 다항식 연산

$n = 7, c(X) = 1 + X^2 + X^3$ 일 때 1칸 순환 이동을 계산한다:

$$X \cdot c(X) = X + X^3 + X^4$$

$\deg(X \cdot c(X)) = 4 < 7$ 이므로 $\text{mod}(X^7 - 1)$ 을 취해도 변하지 않는다. 벡터로 변환하면: $(0, 1, 0, 1, 1, 0, 0) \leftarrow$ 원래 벡터 $(1, 0, 1, 1, 0, 0, 0)$ 을 오른쪽으로 1칸 이동한 것과 일치한다.

예제 2.3: 최고차항이 넘어가는 경우의 순환 이동

$n = 7, c(X) = X^4 + X^5 + X^6$ 일 때 1칸 순환 이동:

$$X \cdot c(X) = X^5 + X^6 + X^7$$

$X^7 \equiv 1 \pmod{X^7 - 1}$ 이므로:

$$X \cdot c(X) \text{ mod } (X^7 - 1) = 1 + X^5 + X^6$$

벡터: $(0, 0, 0, 0, 1, 1, 1) \xrightarrow{1\text{칸 이동}} (1, 0, 0, 0, 0, 1, 1)$ OK

생성 다항식이란 무엇이고, 어떤 성질을 가지는가?

모든 순환 부호는 부호 공간 내에서 차수가 가장 낮고 최고차항의 계수가 1인(Monic) 유일한 다항식 $g(X)$ 에 의해 완전히 결정된다. 이를 생성 다항식(Generator Polynomial)이라 한다.

핵심 성질: - $g(X)$ 는 반드시 $X^n - 1$ 의 인수(Factor)여야 한다: $X^n - 1 = g(X) \cdot h(X)$ - $g(X)$ 의 차수를 $r = n - k$ 라 하면, 부호의 차원(정보 비트 수)은 k 이다. - 모든 부호어 $c(X)$ 는 $g(X)$ 의 배수이다: $c(X) = m(X) \cdot g(X)$

비체계적 인코딩: 메시지 다항식 $m(X)$ (차수 $\leq k - 1$)에 $g(X)$ 를 곱한다:

$$c(X) = m(X) \cdot g(X)$$

예제 3.1: $X^7 - 1$ 의 인수분해와 가능한 생성 다항식

$\text{GF}(2)$ 위에서 $X^7 + 1$ (이진 체에서 $-1 = 1$)을 인수분해하면:

$$X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3)$$

이 세 기약다항식(Irreducible Polynomial)의 조합으로 만들 수 있는 $X^7 + 1$ 의 인수들이 곧 가능한 생성 다항식이다:

$g(X)$	$\deg(g) = n - k$	$[n, k]_2$	부호어 수 2^k
$1 + X$	1	$[7, 6]_2$	64
$1 + X + X^3$	3	$[7, 4]_2$	16
$1 + X^2 + X^3$	3	$[7, 4]_2$	16
$(1 + X)(1 + X + X^3) =$ $1 + X^2 + X^3 + X^4$	4	$[7, 3]_2$	8
$(1 + X)(1 + X^2 + X^3) =$ $1 + X + X^2 + X^4$	4	$[7, 3]_2$	8
$(1 + X + X^3)(1 + X^2 +$ $X^3) = 1 + X + X^2 +$ $X^3 + X^4 + X^5 + X^6$	6	$[7, 1]_2$	2
$X^7 + 1$	7	$[7, 0]_2$	1 (자명)

예제 3.2: 비체계적 인코딩 — $[7, 4]_2$ 부호

$g(X) = 1 + X + X^3$, 메시지 $\mathbf{m} = (1, 1, 0, 1)$, 즉 $m(X) = 1 + X + X^3$

$$\begin{aligned}
c(X) &= m(X) \cdot g(X) \\
&= (1 + X + X^3)(1 + X + X^3) \\
&= 1 + X + X^3 + X + X^2 + X^4 + X^3 + X^4 + X^6 \\
&= 1 + (1 + 1)X + X^2 + (1 + 1)X^3 + (1 + 1)X^4 + X^6 \\
&= 1 + X^2 + X^6
\end{aligned}$$

부호어 벡터: $\mathbf{c} = (1, 0, 1, 0, 0, 0, 1)$

검증: $c(X)$ 가 $g(X)$ 의 배수인지 확인 $\rightarrow (1 + X^2 + X^6) \div (1 + X + X^3)$ 의 나머지가 0이면 정상 OK

예제 3.3: 비체계적 인코딩 — 다른 메시지

$g(X) = 1 + X + X^3$, 메시지 $\mathbf{m} = (0, 1, 0, 0)$, 즉 $m(X) = X$

$$c(X) = X \cdot (1 + X + X^3) = X + X^2 + X^4$$

부호어 벡터: $\mathbf{c} = (0, 1, 1, 0, 1, 0, 0)$

이것은 $g(X)$ 의 계수를 오른쪽으로 1칸 시프트한 것과 동일하다 — 생성 행렬의 두 번째 행에 해당한다.

패리티 검사 다항식이란 무엇인가?

$X^n - 1$ 을 생성 다항식 $g(X)$ 로 나눈 몫을 패리티 검사 다항식(Parity Check Polynomial) $h(X)$ 라 한다:

$$h(X) = \frac{X^n - 1}{g(X)}$$

$h(X)$ 의 차수는 k 이며, 다항식 $v(X)$ 가 유효한 부호어인지 확인하려면:

$$v(X) \cdot h(X) \equiv 0 \pmod{X^n - 1}$$

을 검사한다.

예제 4.1: $[7, 4]_2$ 부호의 패리티 검사 다항식

$g(X) = 1 + X + X^3$ 일 때:

$$h(X) = \frac{X^7 + 1}{1 + X + X^3} = 1 + X + X^2 + X^4$$

검증: $g(X) \cdot h(X) = (1 + X + X^3)(1 + X + X^2 + X^4)$ 를 GF(2)에서 전개:

	1	X	X^2	X^4	
1	1	X	X^2	X^4	
X	X	X^2	X^3	X^5	
X^3	X^3	X^4	X^5	X^7	

합산 (GF(2)에서 같은 항은 소거):

$$= 1 + (1+1)X + (1+1)X^2 + (1+1)X^3 + (1+1)X^4 + (1+1)X^5 + X^7 = 1 + X^7 = X^7 + 1$$

예제 4.2: 부호어 유효성 검사

$c(X) = X + X^2 + X^4$ (예제 3.3의 부호어)에 대해:

$$c(X) \cdot h(X) = (X + X^2 + X^4)(1 + X + X^2 + X^4)$$

$$\begin{aligned} \text{항별 곱: } & - X \cdot (1 + X + X^2 + X^4) = X + X^2 + X^3 + X^5 - X^2 \cdot (1 + X + X^2 + X^4) = X^2 + X^3 + X^4 + X^6 \\ & - X^4 \cdot (1 + X + X^2 + X^4) = X^4 + X^5 + X^6 + X^8 \end{aligned}$$

$$\text{합산 (GF(2)): } - X^1: 1\text{번} \rightarrow X - X^2: 2\text{번} \rightarrow 0 - X^3: 2\text{번} \rightarrow 0 - X^4: 2\text{번} \rightarrow 0 - X^5: 2\text{번} \rightarrow 0 - X^6: 2\text{번} \rightarrow 0 - X^8: 1\text{번} \rightarrow X^8$$

$$\text{결과: } X + X^8$$

$$\text{mod}(X^7 + 1): X^8 \equiv X \text{ 이므로 } X + X = 0 \text{ OK — 유효한 부호어이다.}$$

생성 행렬은 어떻게 만드는가?

생성 다항식 $g(X) = g_0 + g_1X + \dots + g_{n-k}X^{n-k}$ 의 계수를 행마다 오른쪽으로 한 칸씩 시프트하여 $k \times n$ 크기의 생성 행렬 G 를 구성한다:

$$G = \begin{bmatrix} g_0 & g_1 & g_2 & \dots & g_{n-k} & 0 & \dots & 0 \\ 0 & g_0 & g_1 & \dots & g_{n-k-1} & g_{n-k} & \dots & 0 \\ \vdots & & \ddots & & & & \ddots & \vdots \\ 0 & \dots & 0 & g_0 & g_1 & \dots & \dots & g_{n-k} \end{bmatrix}$$

G 의 i 번째 행($i = 0, 1, \dots, k-1$)은 다항식 $X^i \cdot g(X)$ 의 계수 벡터에 해당한다.

왜 이 구조가 유효한 생성 행렬인가? 1. $g(X)$ 는 부호어이고, 순환 이동 불변성에 의해 $Xg(X), X^2g(X), \dots, X^{k-1}g(X)$ 도 모두 부호어이다. 2. 이들의 최고차항 차수가 각각 $n-k, n-k+1, \dots, n-1$ 로 모두 다르므로 **일차 독립**이다. 3. k 차원 부호 공간에서 k 개의 일차 독립 벡터를 찾았으므로 이들이 **기저(Basis)**를 형성한다.

예제 5.1: $[7, 4]_2$ 순환 부호의 생성 행렬

$g(X) = 1 + X + X^3$, 계수 벡터: $(1, 1, 0, 1)$

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

각 행의 의미: - 행 0: $g(X) = 1 + X + X^3$ - 행 1: $Xg(X) = X + X^2 + X^4$ - 행 2: $X^2g(X) = X^2 + X^3 + X^5$ - 행 3: $X^3g(X) = X^3 + X^4 + X^6$

예제 5.2: 생성 행렬을 이용한 인코딩

메시지 $\mathbf{m} = (1, 0, 1, 1)$ 을 인코딩:

$$\mathbf{c} = \mathbf{m} \cdot G = (1, 0, 1, 1) \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

GF(2)에서 계산: - $c_0 = 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 1 \cdot 0 = 1$ - $c_1 = 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 + 1 \cdot 0 = 1$ - $c_2 = 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 = 1$ - $c_3 = 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1 = 1$ - $c_4 = 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 + 1 \cdot 1 = 1$ - $c_5 = 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 = 1$ - $c_6 = 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 0 + 1 \cdot 1 = 1$

$$\mathbf{c} = (1, 1, 1, 1, 1, 1, 1)$$

검증: $c(X) = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6$. 이것이 $g(X)$ 의 배수인지 확인: $(1 + X + X^2 + X^3 + X^4 + X^5 + X^6) \div (1 + X + X^3)$ 의 나머지 = 0 OK

예제 5.3: $(7, 3)$ 순환 부호의 생성 행렬

$g(X) = 1 + X^2 + X^3 + X^4, k = 3$

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

메시지 $\mathbf{m} = (1, 1, 0)$:

$$\mathbf{c} = (1, 1, 0) \cdot G = (1, 1, 1, 0, 0, 1, 0)$$

패리티 검사 행렬은 어떻게 만드는가?

패리티 검사 다항식 $h(X) = h_0 + h_1X + \dots + h_kX^k$ 의 계수를 **역순으로 뒤집은** 후 시프트하여 $(n-k) \times n$ 크기의 패리티 검사 행렬 H 를 구성한다:

$$H = \begin{bmatrix} h_k & h_{k-1} & \dots & h_0 & 0 & \dots & 0 \\ 0 & h_k & h_{k-1} & \dots & h_0 & \dots & 0 \\ \vdots & & \ddots & & & \ddots & \vdots \\ 0 & \dots & 0 & h_k & h_{k-1} & \dots & h_0 \end{bmatrix}$$

이 행렬은 $GH^T = 0$ (직교 조건)을 만족한다.

왜 계수를 역순으로 뒤집는가?

$g(X)h(X) = X^n - 1 \equiv 0 \pmod{X^n - 1}$ 에서, $c(X) = m(X)g(X)$ 인 부호어에 $h(X)$ 를 곱하면:

$$c(X)h(X) = m(X) \cdot g(X) \cdot h(X) = m(X)(X^n - 1) \equiv 0$$

이 다항식 곱의 특정 차수 계수들이 0이 되는 조건을 벡터 내적으로 표현하면, $h(X)$ 의 계수를 역순으로 배치한 벡터와 부호어 벡터의 내적이 0이 된다. 이것이 패리티 검사 방정식 $H\mathbf{c}^T = 0$ 과 정확히 일치한다.

예제 6.1: $[7, 4]_2$ 순환 부호의 패리티 검사 행렬

$h(X) = 1 + X + X^2 + X^4$, 계수: $(1, 1, 1, 0, 1)$

역순: $(1, 0, 1, 1, 1) \rightarrow h_4 = 1, h_3 = 0, h_2 = 1, h_1 = 1, h_0 = 1$

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

예제 6.2: 직교성 검증 $GH^T = 0$

G 의 첫 번째 행 $(1, 1, 0, 1, 0, 0, 0)$ 과 H 의 첫 번째 행 $(1, 0, 1, 1, 1, 0, 0)$ 의 내적:

$$1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 = 1 + 1 = 0 \pmod{2}$$

G 의 첫 번째 행과 H 의 두 번째 행 $(0, 1, 0, 1, 1, 1, 0)$ 의 내적:

$$0 + 1 + 0 + 1 + 0 + 0 + 0 = 0 \pmod{2}$$

(나머지 조합도 모두 0이 된다.)

예제 6.3: 패리티 검사 행렬을 이용한 오류 검출

수신 벡터 $\mathbf{v} = (1, 1, 1, 1, 1, 1, 1)$ (예제 5.2의 부호어)에 대해:

$$H\mathbf{v}^T = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(각 행의 1의 개수가 4개이므로 GF(2)에서 합은 0) → 오류 없음 OK
수신 벡터에 오류가 있는 경우: $\mathbf{v}' = (0, 1, 1, 1, 1, 1, 1)$ (첫 비트 반전)

$$H\mathbf{v}'^T = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \neq \mathbf{0}$$

→ 신드롬이 $(1, 0, 0)$ 으로 오류 검출됨 OK

비체계적 인코딩과 체계적 인코딩의 차이는 무엇인가?

두 방식 모두 유효한 부호어를 생성하지만, 부호어 내에서 원본 메시지의 위치가 다르다.

구분	비체계적 (Non-systematic)	체계적 (Systematic)
인코딩 방법	$c(X) = m(X) \cdot g(X)$	$c(X) = p(X) + X^{n-k}m(X)$
메시지 위치	부호어 전체에 섞여 있음	부호어의 특정 위치에 그대로 보존
디코딩 편의성	나눗셈 필요	패리티 비트만 떼어내면 됨
실무 사용	드물	대부분의 통신 시스템에서 사용

예제 7.1: 같은 메시지의 두 가지 인코딩 비교

$g(X) = 1 + X + X^3$, 메시지 $\mathbf{m} = (1, 0, 1, 1)$

비체계적: $c(X) = m(X) \cdot g(X) = (1 + X^2 + X^3)(1 + X + X^3)$

전개: $1 \cdot (1 + X + X^3) = 1 + X + X^3$ - $X^2 \cdot (1 + X + X^3) = X^2 + X^3 + X^5$ - $X^3 \cdot (1 + X + X^3) = X^3 + X^4 + X^6$

합산: $1 + X + X^2 + (1 + 1 + 1)X^3 + X^4 + X^5 + X^6 = 1 + X + X^2 + X^3 + X^4 + X^5 + X^6$

부호어: $(1, 1, 1, 1, 1, 1, 1)$ — 메시지 $(1, 0, 1, 1)$ 이 어디에 있는지 바로 알 수 없음

체계적 (Q8에서 상세히 다룸): 부호어 $(1, 0, 0, 1, 0, 1, 1)$ — 뒤쪽 4비트가 원본 메시지 그대로

체계적 인코딩은 구체적으로 어떻게 수행하는가?

체계적 인코딩은 3단계로 수행된다:

Step 1. 메시지 시프트: $m(X)$ 에 X^{n-k} 를 곱하여 높은 차수 쪽으로 이동

$$X^{n-k} \cdot m(X)$$

Step 2. 나머지 계산: 시프트된 다항식을 $g(X)$ 로 나누어 나머지 $p(X)$ 를 구함

$$p(X) = X^{n-k}m(X) \bmod g(X)$$

Step 3. 부호어 합성: 시프트된 메시지와 나머지를 더함

$$c(X) = p(X) + X^{n-k}m(X)$$

결과적으로 부호어는 [패리티 $(n - k)$ 비트 | 메시지 k 비트] 구조를 가진다.

왜 이것이 유효한 부호어인가? $c(X) = p(X) + X^{n-k}m(X)$ 에서 $p(X) = X^{n-k}m(X) \bmod g(X)$ 이므로:

$$c(X) = X^{n-k}m(X) - p(X) + 2p(X)$$

GF(2)에서 $2p(X) = 0$ 이고, $X^{n-k}m(X) - p(X)$ 는 $g(X)$ 로 나누어 떨어지므로 $c(X)$ 는 $g(X)$ 의 배수이다. OK

예제 8.1: $[7, 4]_2$ 체계적 인코딩 — 메시지 $(1, 0, 1, 1)$

$$g(X) = 1 + X + X^3, n - k = 3, m(X) = 1 + X^2 + X^3$$

Step 1. $X^3m(X) = X^3 + X^5 + X^6$

Step 2. $(X^6 + X^5 + X^3) \div (X^3 + X + 1)$:

	단계	피제수	몫
1	$X^6 + X^5 + X^3$	$-(X^6 + X^4 + X^3)$	X^3
	$= X^5 + X^4$		
2	$X^5 + X^4$	$-(X^5 + X^3 + X^2)$	X^2
	$= X^4 + X^3 + X^2$		
3	$X^4 + X^3 + X^2$	$-(X^4 + X^2 + X)$	X
	$= X^3 + X$		
4	$X^3 + X$	$-(X^3 + X + 1)$	1
	$= 1$		

몫: $X^3 + X^2 + X + 1$, 나머지: $p(X) = 1$

Step 3. $c(X) = 1 + X^3 + X^5 + X^6$

$$\mathbf{c} = (\underbrace{1, 0, 0}_{\text{패리티}}, \underbrace{1, 0, 1, 1}_{\text{메시지}})$$

예제 8.2: $[7, 4]_2$ 체계적 인코딩 — 메시지 $(1, 1, 0, 0)$

$$m(X) = 1 + X$$

Step 1. $X^3m(X) = X^3 + X^4$

Step 2. $(X^4 + X^3) \div (X^3 + X + 1)$: $-X^4 + X^3 - X(X^3 + X + 1) = X^4 + X^3 - X^4 - X^2 - X = X^3 + X^2 + X - X^3 + X^2 + X - 1 \cdot (X^3 + X + 1) = X^2 + 1$

나머지: $p(X) = X^2 + 1$, 즉 패리티 벡터 $(1, 0, 1)$

Step 3. $c(X) = (1 + X^2) + (X^3 + X^4) = 1 + X^2 + X^3 + X^4$

$$\mathbf{c} = (\underbrace{1, 0, 1}_{\text{패리티}}, \underbrace{1, 1, 0, 0}_{\text{메시지}})$$

예제 8.3: $[7, 4]_2$ 체계적 인코딩 — 메시지 $(0, 0, 0, 1)$

$$m(X) = X^3$$

Step 1. $X^3 \cdot X^3 = X^6$

Step 2. $X^6 \div (X^3 + X + 1): - X^6 - X^3(X^3 + X + 1) = X^6 - X^6 - X^4 - X^3 = X^4 + X^3 -$
 $X^4 + X^3 - X(X^3 + X + 1) = X^3 + X^2 + X - X^3 + X^2 + X - (X^3 + X + 1) = X^2 + 1$

나머지: $p(X) = X^2 + 1$, 패리티 $(1, 0, 1)$

Step 3. $c(X) = (1 + X^2) + X^6 = 1 + X^2 + X^6$

$$\mathbf{c} = (\underbrace{1, 0, 1}_{\text{패리티}}, \underbrace{0, 0, 0, 1}_{\text{메시지}})$$

예제 8.4: $[7, 4]_2$ 전체 체계적 부호어 테이블

메시지 $(m_0m_1m_2m_3)$	$m(X)$	패리티 $(p_0p_1p_2)$	부호어 $(p_0p_1p_2m_0m_1m_2m_3)$
0000	0	000	0000000
1000	1	110	1101000
0100	X	011	0110100
1100	$1 + X$	101	1011100
0010	X^2	111	1110010
1010	$1 + X^2$	001	0011010
0110	$X + X^2$	100	1000110
1110	$1 + X + X^2$	010	0101110
0001	X^3	101	1010001
1001	$1 + X^3$	011	0111001
0101	$X + X^3$	110	1100101
1101	$1 + X + X^3$	000	0001101
0011	$X^2 + X^3$	010	0100011
1011	$1 + X^2 + X^3$	100	1001011
0111	$X + X^2 + X^3$	001	0010111
1111	$1 + X + X^2 + X^3$	111	1111111

신드롬(Syndrome)이란 무엇이고, 오류 검출은 어떻게 하는가?

수신된 다항식 $v(X)$ 를 생성 다항식 $g(X)$ 로 나눈 나머지가 신드롬(Syndrome) $s(X)$ 이다:

$$s(X) = v(X) \bmod g(X)$$

- $s(X) = 0$: 오류 미검출 (유효한 부호어)
- $s(X) \neq 0$: 오류 발생 확인

핵심 원리: 수신 다항식 $v(X) = c(X) + e(X)$ 에서 $c(X)$ 는 $g(X)$ 의 배수이므로:

$$s(X) = v(X) \bmod g(X) = e(X) \bmod g(X)$$

신드롬은 오류 패턴 $e(X)$ 에만 의존하고 원본 메시지와는 무관하다.

예제 9.1: 오류 없는 수신 — 신드롬 = 0

부호어 $c(X) = 1 + X^3 + X^5 + X^6$ 이 그대로 수신됨.

$(X^6 + X^5 + X^3 + 1) \div (X^3 + X + 1)$:

	단계	피제수	몫	항
1		$X^6 + X^5 + X^3 + 1$		X^3
		$-(X^6 + X^4 + X^3)$		
		$= X^5 + X^4 + 1$		
2		$X^5 + X^4 + 1$		X^2
		$-(X^5 + X^3 + X^2)$		
		$= X^4 + X^3 + X^2 + 1$		
3		$X^4 + X^3 + X^2 + 1$		X
		$-(X^4 + X^2 + X)$		
		$= X^3 + X + 1$		
4		$X^3 + X + 1$		1
		$-(X^3 + X + 1)$		
		$= 0$		

신드롬: $s(X) = 0 \rightarrow$ 오류 없음 OK

예제 9.2: 단일 비트 오류 — 위치 X^2

수신: $v(X) = 1 + X^2 + X^3 + X^5 + X^6$ (원래 부호어에 X^2 위치 오류 추가)

오류 다항식: $e(X) = X^2$

$(X^6 + X^5 + X^3 + X^2 + 1) \div (X^3 + X + 1)$:

나눗셈 과정은 예제 9.1과 동일하게 진행하되, X^2 항이 추가되어 있으므로:

신드롬: $s(X) = X^2$, 벡터 $(0, 0, 1)$

$s(X) \neq 0$ 이므로 오류 검출됨. 신드롬 값 X^2 는 오류 위치 X^2 와 대응한다.

예제 9.3: $[7, 4]_2$ 부호의 전체 신드롬 테이블 (단일 오류)

단일 비트 오류 $e(X) = X^i$ 에 대한 신드롬 $s(X) = X^i \bmod g(X)$:

오류 위치 i	$e(X)$	$s(X) = e(X) \bmod (1 + X + X^3)$	신드롬 벡터
0	1	1	$(1, 0, 0)$
1	X	X	$(0, 1, 0)$
2	X^2	X^2	$(0, 0, 1)$
3	X^3	$X^3 \bmod g(X) = 1 + X$	$(1, 1, 0)$
4	X^4	$X^4 \bmod g(X) = X + X^2$	$(0, 1, 1)$
5	X^5	$X^5 \bmod g(X) = 1 + X + X^2$	$(1, 1, 1)$
6	X^6	$X^6 \bmod g(X) = 1 + X^2$	$(1, 0, 1)$

7개의 신드롬이 모두 서로 다르므로, 이 부호는 모든 단일 비트 오류를 정정할 수 있다.

예제 9.4: 신드롬을 이용한 오류 정정 과정

수신 벡터: $\mathbf{v} = (1, 0, 1, 1, 0, 1, 1)$

Step 1. 신드롬 계산: $v(X) = 1 + X^2 + X^3 + X^5 + X^6$

$v(X) \bmod g(X)$ 를 계산하면 $s(X) = X^2 \rightarrow$ 신드롬 벡터 $(0, 0, 1)$

Step 2. 신드롬 테이블 조회: $(0, 0, 1)$ 은 오류 위치 $i = 2$ 에 대응

Step 3. 오류 정정: $\hat{c}(X) = v(X) + X^2 = 1 + X^3 + X^5 + X^6$

정정된 부호어: $\hat{\mathbf{c}} = (1, 0, 0, 1, 0, 1, 1)$

Step 4. 메시지 추출 (체계적 부호): 뒤쪽 4비트 $\rightarrow \mathbf{m} = (1, 0, 1, 1)$ OK

예제 9.5: 2비트 오류의 경우

수신: $\mathbf{v} = (0, 1, 0, 1, 0, 1, 1)$ (위치 0과 1에 동시 오류)

$e(X) = 1 + X$, 신드롬: $s(X) = (1 + X) \bmod (1 + X + X^3) = 1 + X$, 벡터 $(1, 1, 0)$

신드롬 테이블에서 $(1, 1, 0)$ 은 위치 3의 단일 오류에 대응한다. 따라서 수신기는 위치 3을 “정정”하려 하지만, 실제로는 위치 0과 1에 오류가 있으므로 **오정정(Miscorrection)**이 발생한다.

이것이 $[7, 4]_2$ 부호의 한계이다: 최소 거리 $d_{\min} = 3$ 이므로 **1비트 오류 정정** 또는 **2비트 오류 검출**만 가능하다.

순환 부호의 디코딩(복호) 과정은 어떻게 되는가?

순환 부호의 디코딩은 부호의 특성(단순 순환 부호, BCH 부호, RS 부호 등)에 따라 세부적인 구현이 다르지만, 공통적으로 다음 4단계의 핵심 과정을 거친다.

Step 1: 신드롬 계산 (Syndrome Calculation)

수신 다항식 $v(X) = c(X) + e(X)$ 를 파악하고, 생성 다항식 $g(X)$ 로 나누어 나머지를 구한다.

$$s(X) = v(X) \bmod g(X)$$

이 과정은 수신되는 비트 스트림을 LFSR(Linear Feedback Shift Register) 회로에 순차적으로 입력하여 매우 빠르게(실시간으로) 처리할 수 있다.

Step 2: 오류 패턴 탐색 (Error Pattern Search)

계산된 신드롬 $s(X)$ 를 바탕으로 실제 발생한 오류 다항식 $e(X)$ 를 찾아낸다. 여기에서 순환 부호만의 특수한 기법들이 사용된다. - **룩업 테이블 (Syndrome Table)**: 짧은 블록 부호(예: Golay 코드, 작은 Hamming 코드)에서는 미리 배정된 신드롬-오류 패턴 테이블을 이용해 $e(X)$ 를 직접 찾는다. - **메깃 디코더 (Meggitt Decoder)**: 하드웨어 구현에 매우 친화적인 방식이다. 신드롬 레지스터를 1클럭씩 회전(Shift)시키면서 특정 조건(최상위 비트에 오류가 있는 패턴)이 만족되는지 회로를 통해 감지한다. 조건이 맞으면 해당 비트를 반전시켜 정정한다. - **대수적 디코딩 (Algebraic Decoding)**: 다중 오류를 정정할 수 있는 BCH 부호나 Reed-Solomon 부호에서는 룩업 테이블 적용이 불가능하다. 이 경우에는 **Berlekamp-Massey 알고리즘** 등을 사용하여 오류 위치 다항식을 구성하고, **Chien Search**를 통해 오류 위치를 특정하며, **Forney 알고리즘**을 통해 오류 값을 직접 계산한다.

Step 3: 오류 정정 (Error Correction)

찾아낸 오류 패턴 $\hat{e}(X)$ 를 수신 벡터 $v(X)$ 에서 감산하여 정정된 부호어 $\hat{c}(X)$ 를 획득한다. 모듈로-2 덧셈(XOR)을 사용하는 이진 부호의 경우 감산은 덧셈과 동일하다.

$$\hat{c}(X) = v(X) - \hat{e}(X) \quad (\text{또는 이진 체에서 } v(X) + \hat{e}(X))$$

Step 4: 메시지 추출 (Message Extraction)

정상적인 부호어 $\hat{c}(X)$ 를 얻은 뒤, 원본 메시지 $m(X)$ 를 복원한다. - **체계적(Systematic) 부호**: 부호어의 특정 위치(대개 끝부분)에 위치한 메시지 비트들만 추출하면 된다. 실무에서는 주로 이 방법을 사용한다. - **비체계적(Non-systematic) 부호**: $\hat{c}(X)$ 를 $g(X)$ 로 나누어 그 몫을 계산함으로써 $m(X)$ 를 얻는다.

순환 부호의 오류 정정 능력은 어떻게 결정되는가?

순환 부호도 선형 부호의 일종이므로, 오류 정정 능력은 **최소 거리(Minimum Distance) $d_{\min}$** 에 의해 결정된다. 순환 부호에만 특화된 별도의 오류 정정 공식은 없으며, 모든 선형 부호에 공통으로 적용되는 다음 관계를 따른다.

핵심 정리: t -오류 정정 능력

최소 거리가 $d_{\min}$ 인 부호는 최대 t 개의 오류를 정정할 수 있다:

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

또한 오류 **검출**만 수행하는 경우에는 최대 $d_{\min} - 1$ 개의 오류를 검출할 수 있다.

예제 11.1: 대표적인 순환 부호의 오류 정정 능력

부호	$[n, k]$	$d_{\min}$	정정 능력 t	검출 능력
$[7, 4]$ 순환 부호 (Hamming)	$[7, 4]_2$	3	1	2
$[15, 7]$ BCH 부호	$[15, 7]$	5	2	4
$[23, 12]$ Golay 부호	$[23, 12]$	7	3	6
$[15, 5]$ BCH 부호	$[15, 5]$	7	3	6
$[31, 21]$ BCH 부호	$[31, 21]$	5	2	4

일반 순환 부호의 한계

일반적인 순환 부호에서는 생성 다항식 $g(X)$ 만으로 $d_{\min}$ 을 이론적으로 바로 구하는 공식이 존재하지 않는다. 부호어를 직접 열거하거나 대수적 방법으로 계산해야 한다.

BCH 한계 (BCH Bound): 설계 거리의 보장

순환 부호의 하위 클래스인 **BCH 부호**에는 최소 거리의 하한을 보장하는 유명한 정리가 있다.

BCH 한계 정리: α 를 $\text{GF}(q^m)$ 에서 $X^n - 1 = 0$ 의 원시 n -차 단위근이라 하자. 생성 다항식 $g(X)$ 가 연속 $\delta - 1$ 개의 거듭제곱근

$$\alpha^b, \alpha^{b+1}, \dots, \alpha^{b+\delta-2}$$

을 근으로 가지면, 부호의 최소 거리는 다음을 만족한다:

$$d_{\min} \geq \delta$$

여기서 δ 를 **설계 거리(Designed Distance)**라 부른다. 이 정리 덕분에 BCH 부호를 설계할 때 원하는 오류 정정 능력 t 를 미리 지정할 수 있다: $\delta = 2t + 1$ 이 되도록 연속근의 개수를 선택하면 된다.

예제 11.2: BCH 한계 적용

(15, 7) 이진 BCH 부호에서 α 를 $\text{GF}(2^4)$ 의 원시 15차 단위근이라 하면, 생성 다항식이 $\alpha, \alpha^2, \alpha^3, \alpha^4$ 를 근으로 가집니다 (연속 4개, $b = 1$).

$$\delta = 4 + 1 = 5 \Rightarrow d_{\min} \geq 5 \Rightarrow t \geq 2$$

실제로 이 부호의 $d_{\min} = 5$ 이므로 BCH 한계가 정확히 달성된다.

예제 11.3: 일반 순환 부호 vs BCH 부호

$[7, 4]_2$ 순환 부호에서 $g(X) = 1 + X + X^3$ 의 경우: - $\text{GF}(2^3)$ 에서 α 를 $X^7 + 1$ 의 원시근이라 하면, $g(\alpha) = 0, g(\alpha^2) = 0$ 이므로 α, α^2 가 연속근 (2개, $b = 1$) - BCH 한계: $\delta = 3$, 즉 $d_{\min} \geq 3$ - 실제 $d_{\min} = 3$ 이므로 한계가 정확히 달성됨 - 이 부호는 사실 $[7, 4]_2$ Hamming 부호이자 BCH 부호이기도 하다
이처럼 BCH 부호는 설계 단계에서 오류 정정 능력을 제어할 수 있다는 점이 일반 순환 부호 대비 큰 장점이며, 실무에서 BCH/RS 부호가 선호되는 핵심 이유이다.

순환 부호의 대수적 구조는 무엇인가?

순환 부호는 다항식 몫환(Quotient Ring) $R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle$ 의 주 아이디얼(Principal Ideal)이다.

이것은 다음을 의미한다: 1. 순환 부호 $\mathcal{C}$ 는 환 R_n 의 아이디얼(Ideal)이다. 2. 이 아이디얼은 단 하나의 원소 $g(X)$ 로 생성된다: $\mathcal{C} = \langle g(X) \rangle$

증명 개요

Step 1: $\mathcal{C}$ 가 아이디얼임을 보임

- 덧셈 닫힘: 선형성에 의해 $a(X), b(X) \in \mathcal{C} \Rightarrow a(X) + b(X) \in \mathcal{C}$
- 흡수성: 임의의 $r(X) \in R_n$ 과 $c(X) \in \mathcal{C}$ 에 대해 $r(X) \cdot c(X) \in \mathcal{C}$
 - 순환 이동 불변성: $X \cdot c(X) \in \mathcal{C}$
 - 반복 적용: $X^2 c(X), X^3 c(X), \dots \in \mathcal{C}$
 - 선형성: $r(X) = \sum a_i X^i$ 이므로 $r(X)c(X) = \sum a_i X^i c(X) \in \mathcal{C}$

Step 2: 주 아이디얼임을 보임

- $\mathcal{C}$ 에서 차수가 가장 낮은 monic 다항식을 $g(X)$ 라 정의
- 임의의 $c(X) \in \mathcal{C}$ 에 대해 나눗셈 알고리즘 적용: $c(X) = q(X)g(X) + r(X), \deg(r) < \deg(g)$
- $r(X) = c(X) - q(X)g(X) \in \mathcal{C}$ 이고, $\deg(r) < \deg(g)$ 이면 $g(X)$ 의 최소 차수 조건에 모순
- 따라서 $r(X) = 0$, 즉 모든 $c(X) = q(X)g(X)$
- 결론: $\mathcal{C} = \langle g(X) \rangle$

예제 10.1: 아이디얼 구조 확인

$[7, 4]_2$ 부호에서 $g(X) = 1 + X + X^3$:

- $g(X) \in \mathcal{C}$ OK
- $X \cdot g(X) = X + X^2 + X^4 \in \mathcal{C}$ OK (순환 이동)
- $(1 + X) \cdot g(X) = (1 + X)(1 + X + X^3) = 1 + X^2 + X^3 + X^4 \in \mathcal{C}$ OK (흡수성)

예제 10.2: $g(X)$ 가 $X^n - 1$ 의 인수임을 확인

$g(X) = 1 + X + X^3$ 이 $X^7 + 1$ 을 나누는지 확인:

$$X^7 + 1 = (1 + X + X^3)(1 + X + X^2 + X^4) = g(X) \cdot h(X)$$

나머지가 0이므로 $g(X) \mid (X^7 + 1)$ OK

이것은 필수 조건이다: 생성 다항식은 반드시 $X^n - 1$ 의 인수여야 한다.

몫환의 이데알 개수와 생성 다항식의 결정

앞 절에서 순환 부호가 몫환 $R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle$ 의 주 이데알(Principal Ideal)과 정확히 일대일 대응함을 보았다. 이로부터 자연스럽게 두 가지 질문이 따라온다: (1) 이데알은 총 몇 개인가? (2) 각 이데알을 정의하는 생성 다항식은 어떻게 결정되는가?

핵심 정리: 이데알의 개수

R_n 의 모든 이데알은 $X^n - 1$ 의 monic 인수 $g(X)$ 가 생성하는 주 이데알 $\langle g(X) \rangle$ 이다.

증명.

$R_n = \mathbb{F}_q[X]/\langle X^n - 1 \rangle$ 의 임의의 이데알 I 에 대해, I 가 어떤 monic $g(X) \mid (X^n - 1)$ 에 의해 $I = \langle g(X) \rangle$ 로 표현됨을 보인다.

Step 1. 자연스러운 전사 준동형사상 구성. 표준 사영(canonical projection) $\pi : \mathbb{F}_q[X] \rightarrow R_n$, $f(X) \mapsto f(X) + \langle X^n - 1 \rangle$ 을 생각하자. π 는 환 전사 준동형사상이고, $\ker(\pi) = \langle X^n - 1 \rangle$ 이다.

Step 2. 대응 정리(Correspondence Theorem) 적용. 환의 대응 정리에 의해, R_n 의 이데알 I 와 $\langle X^n - 1 \rangle \subseteq J$ 를 만족하는 $\mathbb{F}_q[X]$ 의 이데알 J 사이에 일대일 대응이 존재한다:

$$I = J/\langle X^n - 1 \rangle, \quad J = \pi^{-1}(I)$$

Step 3. $\mathbb{F}_q[X]$ 는 주 이데알 정역(PID)이다. $\mathbb{F}_q[X]$ 는 유클리드 정역이므로 모든 이데알이 주 이데알이다. 따라서 $J = \langle g(X) \rangle$ 인 monic 다항식 $g(X)$ 가 유일하게 존재한다.

Step 4. $g(X)$ 가 $X^n - 1$ 의 인수임을 보임. $\langle X^n - 1 \rangle \subseteq \langle g(X) \rangle$ 이므로 $g(X) \mid (X^n - 1)$ 이다.

Step 5. 역방향 — $X^n - 1$ 의 모든 monic 인수가 이데알을 생성함. 역으로, $g(X) \mid (X^n - 1)$ 이면 $\langle X^n - 1 \rangle \subseteq \langle g(X) \rangle$ 이므로 대응 정리에 의해 $\langle g(X) \rangle = \langle g(X) \rangle / \langle X^n - 1 \rangle$ 은 R_n 의 이데알이다.

결론. R_n 의 이데알과 $X^n - 1$ 의 monic 인수 사이에 일대일 대응이 성립한다:

$$g(X) \mid (X^n - 1) \iff \langle g(X) \rangle \text{은 } R_n \text{의 이데알}$$

■

따라서 R_n 의 이데알 개수 = $\mathbb{F}_q[X]$ 위에서 $X^n - 1$ 의 monic 인수의 개수이다.

$\gcd(n, q) = 1$ 인 경우 (즉, $X^n - 1$ 이 중복근을 갖지 않는 경우), $X^n - 1$ 이 $\mathbb{F}_q[X]$ 위에서 s 개의 서로 다른 기약다항식의 곱으로 분해된다면:

$$X^n - 1 = f_1(X) \cdot f_2(X) \cdots f_s(X)$$

각 기약인수를 “포함하거나 포함하지 않거나” 독립적으로 선택할 수 있으므로, monic 인수의 총 개수는:

$$\text{이데알의 개수} = 2^s$$

이 2^s 개에는 $g(X) = 1$ 인 자명한 전체 환(= $\mathbb{F}_q^n$ 전체)과 $g(X) = X^n - 1$ 인 영 이데알(= $\{0\}$)이 모두 포함된다.

왜 2^s 인가? $X^n - 1$ 의 monic 인수는 기약인수 $f_1, f_2, \dots, f_s$ 의 부분집합 $S \subseteq \{1, 2, \dots, s\}$ 에 대해 $g_S(X) = \prod_{i \in S} f_i(X)$ 로 유일하게 결정된다. 부분집합의 개수가 2^s 이므로 인수도 2^s 개이다. 이것은 인수 격자(Divisor Lattice)와 멱집합(Power Set) 사이의 동형에 해당한다.

생성 다항식의 결정 절차

각 이데알(= 순환 부호)을 정의하는 생성 다항식 $g(X)$ 는 다음 절차로 결정된다:

Step 1. $X^n - 1$ 을 $\mathbb{F}_q[X]$ 위에서 기약다항식으로 완전히 인수분해한다.

Step 2. 기약인수들의 모든 부분집합 조합의 곱이 각각 하나의 생성 다항식 $g(X)$ 가 된다.

Step 3. 각 $g(X)$ 가 정의하는 순환 부호의 파라미터는 $(n, n - \deg g(X))$ 이다.

예제 13.1: $n = 7, \mathbb{F}_2$ 위의 전체 이데알 열거

예제 3.1에서 $X^7 + 1 = (1 + X)(1 + X + X^3)(1 + X^2 + X^3)$ 이므로 기약인수 $s = 3$ 개이다.

이데알의 개수: $2^3 = 8$ 개

부분집합 S	$g(X) = \prod_{i \in S} f_i(X)$	$\deg(g)$	$[n, k]$	부호어 수 2^k	비고
$\emptyset$	1	0	(7, 7)	128	전체 환 (자명)
$\{1\}$	$1 + X$	1	(7, 6)	64	단순 패리티 검사
$\{2\}$	$1 + X + X^3$	3	$[7, 4]_2$	16	Hamming (7, 4)
$\{3\}$	$1 + X^2 + X^3$	3	$[7, 4]_2$	16	
$\{1, 2\}$	$1 + X^2 + X^3 + X^4$	4	(7, 3)	8	반복 부호
$\{1, 3\}$	$1 + X + X^2 + X^4$	4	(7, 3)	8	
$\{2, 3\}$	$1 + X + X^2 + X^3 + X^4 + X^5 + X^6$	6	(7, 1)	2	
$\{1, 2, 3\}$	$X^7 + 1$	7	(7, 0)	1	영 이데알 (자명)

이 8개의 이데알은 포함 관계에 의해 격자(Lattice) 구조를 형성한다: $g_1(X) \mid g_2(X)$ 이면 $\langle g_2(X) \rangle \subseteq \langle g_1(X) \rangle$ 이다. 즉, 생성 다항식의 차수가 클수록 부호의 차원은 작아지고, 이데알은 더 작은 부분공간이 된다.

예제 13.2: $n = 15, \mathbb{F}_2$ 위의 이데알 개수

연습문제 2에서 $X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4)$ 이므로 기약인수 $s = 5$ 개이다.

이데알의 개수: $2^5 = 32$ 개

이 32개의 이데알 중에서 차수 4인 생성 다항식을 갖는 $(15, 11)$ 부호만 해도 여러 개가 존재한다 — 차수 4인 기약인수 하나를 선택하거나, 차수가 합쳐서 4가 되는 기약인수 조합을 선택하면 된다.

예제 13.3: 이데알 격자와 포함 관계

$n = 7$ 에서 이데알 간의 포함 관계를 격자 다이어그램으로 나타내면:

$$\begin{array}{c}
 \langle 1 \rangle = R_7 \quad (\text{전체 환}, k=7) \\
 / \quad | \quad \backslash \\
 \langle f_1 \rangle \quad \langle f_2 \rangle \quad \langle f_3 \rangle \quad (k=6, 4, 4) \\
 | \backslash \quad / \quad \backslash \quad / \\
 \langle f_1 f_2 \rangle \quad \langle f_1 f_3 \rangle \quad \langle f_2 f_3 \rangle \quad (k=3, 3, 1) \\
 \backslash \quad | \quad / \\
 \langle f_1 f_2 f_3 \rangle = \langle 0 \rangle \quad (\text{영 이데알}, k=0)
 \end{array}$$

위로 갈수록 이데알이 크고(부호어가 많고), 아래로 갈수록 이데알이 작다(부호어가 적지만 오류 정정 능력이 높다).

$\gcd(n, q) \neq 1$ 인 경우의 주의사항

$\gcd(n, q) \neq 1$ 이면 $X^n - 1$ 이 $\mathbb{F}_q[X]$ 위에서 중복근을 가진다. 예를 들어 연습문제 6의 $X^6 + 1$ over $\mathbb{F}_2$ 에서는 $\gcd(6, 2) = 2$ 이므로:

$$X^6 + 1 = (1 + X)^2(1 + X + X^2)^2$$

이 경우 기약인수 $f_i(X)$ 의 거듭제곱 $f_i(X)^{e_i}$ 에서 $0 \leq e'_i \leq e_i$ 를 각각 독립적으로 선택할 수 있으므로, monic 인수의 개수는:

$$\prod_{i=1}^s (e_i + 1)$$

위 예에서는 $(2 + 1)(2 + 1) = 9$ 개의 이데알이 존재한다. 이는 $2^s = 4$ 보다 많다.

순환 부호가 실무에서 중요한 이유는 무엇인가?

순환 부호의 가장 큰 실무적 장점은 LFSR(Linear Feedback Shift Register)을 이용한 초고속 실시간 하드웨어 구현이 가능하다는 점이다.

LFSR 구조의 원리

일반 선형 블록 부호는 인코딩 시 거대한 행렬 곱셈이 필요하지만, 순환 부호의 다항식 나눗셈은 LFSR 회로로 극도로 가볍게 구현된다:

- $g(X)$ 의 차수 r 개의 플립플롭(Flip-flop) (1비트 저장소)
- 다항식 계수에 맞춘 몇 개의 XOR 게이트
- 이것만으로 회로 구성 완료

실시간 처리의 장점

일반 선형 부호	순환 부호 (LFSR)
전체 블록을 버퍼에 저장 후 행렬 곱셈	비트 단위 실시간(On-the-fly) 처리
대용량 메모리 필요	레지스터 r 개만 필요
높은 지연(Latency)	거의 0에 가까운 지연
복잡한 병렬 연산 게이트	단순한 직렬 XOR 회로

실전 예시: CRC와 네트워크 전송

USB, 이더넷(Ethernet) 등에서 기가비트급 데이터 스트림의 무결성을 검사할 때:

1. **메모리 불필요**: 데이터를 모아둘 필요 없이 비트가 들어오는 대로 처리
2. **On-the-fly 연산**: 시스템 클럭에 맞춰 LFSR에 1비트씩 입력
3. **즉시 결과**: 마지막 비트가 통과하는 순간, LFSR에 남은 r 비트가 곧 패리티/신드롬

예제 11.1: $g(X) = 1 + X + X^3$ 에 대한 LFSR 인코더 동작

체계적 인코딩용 LFSR은 $g(X) = 1 + X + X^3$ 의 계수에 따라 구성된다. 3개의 플립플롭 $[R_0, R_1, R_2]$ 를 사용하며, 피드백은 입력 비트와 R_2 (최상위 레지스터)의 XOR로 계산된다. $g(X)$ 에서 X^0 과 X^1 의 계수가 1이므로, 피드백 값이 R_0 과 R_1 에 XOR로 더해진다.

메시지 비트 $(1, 0, 1, 1)$ 을 **높은 차수부터** (m_3, m_2, m_1, m_0 순서로) 입력한다:

클럭	입력 비트	피드백 = 입력 $\oplus R_2$	R_0	R_1	R_2
초기	-	-	0	0	0
1	$m_3 = 1$	$1 \oplus 0 = 1$	1	0	0
2	$m_2 = 1$	$1 \oplus 0 = 1$	1	1	0
3	$m_1 = 0$	$0 \oplus 0 = 0$	0	1	1
4	$m_0 = 1$	$1 \oplus 1 = 0$	0	0	1

4클럭 후 레지스터 상태: $(R_0, R_1, R_2) = (0, 0, 1)$

검증: 예제 8.1에서 $m(X) = 1 + X^2 + X^3$ 의 체계적 인코딩 나머지는 $p(X) = 1$, 즉 패리티 벡터 $(1, 0, 0)$ 이었다. LFSR의 출력 비트 순서는 회로 구성 방식(MSB-first vs LSB-first, 레지스터 번호 매핑)에 따라 달라지며, 위 결과 $(0, 0, 1)$ 을 역순으로 읽으면 $(1, 0, 0)$ 과 일치한다.

참고: LFSR 회로의 구체적인 동작은 피드백 연결 방식과 비트 입력 순서에 따라 달라진다. 핵심은 $n - k$ 개의 레지스터와 몇 개의 XOR 게이트만으로 다항식 나눗셈을 실시간 수행할 수 있다는 점이다.

대표적인 순환 부호에는 어떤 것들이 있는가?

순환 부호의 특성을 가지는 주요 부호들:

부호	특징	주요 응용
CRC (Cyclic Redundancy Check) BCH 부호	오류 검출 전용, 다양한 생성 다항식 표준 갈루아 확대체에서 근을 지정하여 오류 정정 능력 보장	이더넷, USB, ZIP, 저장 매체 플래시 메모리, 위성 통신
Reed-Solomon (RS) 부호 일부 Hamming 부호	비이진 순환 부호, 심볼 단위 오류 정정 특정 파라미터에서 순환 부호 형태 가능	CD/DVD/블루레이, QR코드, 심우주 통신 ECC 메모리

예제 15.1: CRC-8 생성 다항식

CRC-8 표준: $g(X) = X^8 + X^2 + X + 1$

- 부호 길이: 가변 (메시지 길이에 따라)
- 패리티 비트: 8비트
- 용도: 센서 네트워크, 임베디드 시스템

예제 15.2: CRC-32 (이더넷)

$g(X) = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$

- 32비트 패리티로 최대 $2^{32} - 1$ 길이의 프레임 보호
- 모든 이더넷 프레임의 FCS(Frame Check Sequence)에 사용

준순환 부호와 양자내성암호(HQC)는 무엇인가?

순환 부호의 대수적 성질은 현대 암호학, 특히 양자내성암호(PQC: Post-Quantum Cryptography)의 핵심 기반으로 활용된다.

준순환 부호 (Quasi-Cyclic Code, QC)

순환 부호가 1칸 시프트에 대해 닫혀있다면, 준순환 부호는 l 칸 시프트에 대해 닫혀있는 일반화된 구조이다.

- 부호어 길이 $n = m \cdot l$ 일 때, l 칸 순환 시프트한 결과가 부호어 공간에 보존
- $l = 1$ 이면 순환 부호와 동일
- 생성/패리티 검사 행렬이 $m \times m$ 순환 행렬 블록(Circulant Block)들로 구성

HQC (Hamming Quasi-Cyclic)에서의 활용

양자 컴퓨터의 쇼어 알고리즘에 안전한 공개키 캡슐화 메커니즘(KEM)인 HQC는 준순환 부호를 핵심적으로 활용한다.

기존 문제: McEliece 암호 시스템은 보안성은 뛰어나지만 공개키 크기가 ~1MB로 방대

HQC의 해결책: 준순환 구조를 채택하여:

1. 공개키 크기 감소: 순환 행렬은 첫 번째 행(다항식 1개)만으로 전체 행렬 복원 가능 → 수 KB 수준으로 축소

2. **고속 연산**: 다항식 환 $\mathbb{F}_2[X]/\langle X^n - 1 \rangle$ 위의 연산으로 FFT 등 고속 알고리즘 적용 가능
3. **증명 가능한 안전성**: 결정적 신드롬 디코딩 문제(DSDP)와의 수학적 동치(Reduction) 성립 $\rightarrow$ 양자 컴퓨터로도 효율적 공격 불가

예제 16.1: 순환 행렬 블록의 구조

첫 번째 행이 $(1, 0, 1, 1)$ 인 4×4 순환 행렬:

$$C = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}$$

각 행은 이전 행을 오른쪽으로 1칸 순환 이동한 것이다. 따라서 **첫 행 하나만 저장하면 전체 행렬을 복원할 수 있다** — 이것이 공개키 크기를 획기적으로 줄이는 원리이다.

연습문제

연습 1. 순환 부호 판별

다음 부호가 순환 부호인지 판별하시오. ($n = 5$)

$$\mathcal{C} = \{00000, 10110, 01011, 11101, 01101, 10011, 00111, 11010, 11001, 00101, \dots\}$$

힌트: 각 부호어를 1칸 순환 이동시켜 결과가 부호어 집합에 속하는지 확인하라.

연습 2. 생성 다항식 찾기

$\text{GF}(2)$ 위에서 $X^{15} + 1$ 의 기약다항식 인수분해가 다음과 같을 때:

$$X^{15} + 1 = (1 + X)(1 + X + X^2)(1 + X + X^4)(1 + X^3 + X^4)(1 + X + X^2 + X^3 + X^4)$$

- (15, 11) 순환 부호를 만들 수 있는 차수 4인 생성 다항식을 모두 나열하시오.
- $g(X) = 1 + X + X^4$ 를 생성 다항식으로 사용할 때, 대응하는 패리티 검사 다항식 $h(X)$ 를 구하시오.

연습 3. 체계적 인코딩 실습

$[7, 4]_2$ 순환 부호, $g(X) = 1 + X + X^3$ 에 대해 다음 메시지들을 체계적으로 인코딩하시오:

- $\mathbf{m} = (0, 1, 1, 0)$
- $\mathbf{m} = (1, 1, 1, 1)$
- 각 결과가 예제 8.4의 부호어 테이블과 일치하는지 확인하시오.

연습 4. 신드롬 계산과 오류 정정

$[7, 4]_2$ 순환 부호에서 다음 수신 벡터에 대해 신드롬을 계산하고, 단일 비트 오류가 있다면 정정하시오:

(a) $\mathbf{v} = (1, 1, 0, 1, 0, 0, 0)$

(b) $\mathbf{v} = (0, 1, 1, 0, 1, 0, 1)$

(c) $\mathbf{v} = (1, 0, 1, 1, 1, 0, 0)$

힌트: 예제 9.3의 신드롬 테이블을 활용하라.

연습 5. 생성 행렬과 패리티 검사 행렬

$g(X) = 1 + X^2 + X^3$ 을 생성 다항식으로 하는 $[7, 4]_2$ 순환 부호에 대해:

(a) 비체계적 생성 행렬 G 를 구하시오.

(b) 패리티 검사 다항식 $h(X)$ 를 구하시오.

(c) 패리티 검사 행렬 H 를 구하시오.

(d) $GH^T = 0$ 임을 검증하시오.

연습 6. 대수적 구조 이해

(a) $\text{GF}(2)$ 위에서 $X^6 + 1$ 을 인수분해하시오.

주의: $\gcd(6, 2) = 2 \neq 1$ 이므로 $X^6 + 1$ 은 $\text{GF}(2)$ 위에서 중복근을 가진다 (즉, 제곱인수가 존재한다). 이 경우 표준적인 순환 부호 이론의 일부 성질(예: $X^n - 1$ 의 서로 다른 기약인수로의 분해)이 그대로 적용되지 않을 수 있음에 유의하시오. 인수분해 결과에서 중복인수를 확인하시오.

(b) 위 인수분해를 바탕으로, $X^6 + 1$ 의 인수가 될 수 있는 다항식들을 나열하고, 대응하는 $[n, k]$ 파라미터를 구하시오.

(c) 차원이 가장 큰 부호의 생성 다항식과 최소 거리를 구하시오.

연습 7. 종합 문제

$(15, 7)$ BCH 부호의 생성 다항식이 $g(X) = 1 + X^4 + X^6 + X^7 + X^8$ 일 때:

(a) 이 부호가 순환 부호임을 확인하시오 ($g(X)$ 가 $X^{15} + 1$ 을 나누는지 검증).

(b) 메시지 $\mathbf{m} = (1, 0, 0, 0, 0, 0, 1)$ 을 체계적으로 인코딩하시오.

(c) 인코딩된 부호어의 신드롬이 0임을 확인하시오.

SageMath 실습 코드

아래 SageMath 코드를 실행하면 본 문서에서 다룬 순환 부호의 핵심 개념들을 직접 확인할 수 있다.

```
# =====  
# 순환 부호 (Cyclic Code) — SageMath 실습 코드  
# 본 문서(Cyclic_Codes.md)의 예제들을 검증한다.  
# =====
```

```

F = GF(2)
R.<X> = PolynomialRing(F)

# =====
# 1.  $X^7 + 1$  인수분해 (예제 3.1)
# =====
print("=" * 70)
print("1.  $X^7 + 1$  의 GF(2) 위 인수분해")
print("=" * 70)

f = X^7 + 1
print(f" $X^7 + 1 = \{factor(f)\}$ ")
print()

# 세 기약다항식
p1 = 1 + X
p2 = 1 + X + X^3
p3 = 1 + X^2 + X^3

print("기약다항식 확인:")
print(f"  $(1+X)(1+X+X^3)(1+X^2+X^3) = \{p1 * p2 * p3\}$ ")
print(f"  $X^7 + 1$  과 일치: {'OK' if p1 * p2 * p3 == f else 'X'}")
print()

# 가능한 모든 생성 다항식 나열
print("가능한 생성 다항식 g(X)과 대응하는 [n, k]:")
print("-" * 50)
irreducibles = [p1, p2, p3]
from itertools import combinations
all_divisors = [R(1)]
for r in range(1, 4):
    for combo in combinations(irreducibles, r):
        prod = R(1)
        for p in combo:
            prod *= p
        all_divisors.append(prod)
all_divisors.append(f)
all_divisors.sort(key=lambda p: p.degree())

for g in all_divisors:
    deg = g.degree()
    k = 7 - deg
    if 0 <= k <= 7:
        print(f"  $g(X) = \{g\}$ ,  $deg = \{deg\}$ ,  $(7, \{k\})$ , 부호어 수 =  $\{2^k\}$ ")
print()

# =====
# 2. (7,4) 순환 부호 생성 —  $g(X) = 1 + X + X^3$ 
# =====
print("=" * 70)
print("2. [7, 4]2 순환 부호:  $g(X) = 1 + X + X^3$ ")
print("=" * 70)

```

```

g = 1 + X + X^3
n = 7
k = n - g.degree()

# SageMath 내장 순환 부호 생성
C = codes.CyclicCode(generator_pol=g, length=n)
print(f"부호 파라미터: [{C.length()}, {C.dimension()}, {C.minimum_distance()}]")
print(f"생성 다항식: g(X) = {g}")
print()

# 패리티 검사 다항식 (예제 4.1)
h = (X^7 + 1) // g
print(f"패리티 검사 다항식: h(X) = {h}")
print(f"검증: g(X) * h(X) = {g * h}  (== X^7+1: {'OK' if g * h == f else 'X'})")
print()

# =====
# 3. 생성 행렬과 패리티 검사 행렬
# =====

print("=" * 70)
print("3. 생성 행렬 G 와 패리티 검사 행렬 H")
print("=" * 70)

# 비체계적 생성 행렬: 행 i = X^i * g(X)의 계수
G = Matrix(F, k, n)
for i in range(k):
    poly = (X^i * g) % (X^n + 1)
    for j in range(n):
        G[i, j] = poly[j]

print("비체계적 생성 행렬 G:")
print(G)
print()

# 패리티 검사 행렬: h(X)의 계수를 역순으로 뒤집어 시프트
h_coeffs = [h[k - j] for j in range(k + 1)] # 역순 계수
H = Matrix(F, n - k, n)
for i in range(n - k):
    for j in range(k + 1):
        H[i, i + j] = h_coeffs[j]

print("패리티 검사 행렬 H:")
print(H)
print()

# 직교성 검증 (예제 6.2)
product = G * H.transpose()
print(f"G * H^T = 0 검증: {'OK' if product == 0 else 'X'}")
print(G * H.transpose())
print()

# =====
# 4. 비체계적 인코딩 (예제 3.2, 3.3)
# =====

```

```

print("=" * 70)
print("4. 비체계적 인코딩:  $c(X) = m(X) * g(X)$ ")
print("=" * 70)

test_messages_nonsys = [
    ("(1,1,0,1)", 1 + X + X^3),
    ("(0,1,0,0)", X),
    ("(1,0,1,1)", 1 + X^2 + X^3),
]

for label, m_poly in test_messages_nonsys:
    c_poly = (m_poly * g) % (X^n + 1)
    c_vec = vector(F, [c_poly[i] for i in range(n)])
    print(f" m = {label}, m(X) = {m_poly}")
    print(f" c(X) = {c_poly}, c = {list(c_vec)}")
    # g(X)의 배수인지 확인
    remainder = c_poly % g
    print(f" c(X) mod g(X) = {remainder} ({'OK g(X)의 배수' if remainder == 0 else 'X'})")
    print()

# =====
# 5. 체계적 인코딩 (예제 8.1~8.3)
# =====

print("=" * 70)
print("5. 체계적 인코딩:  $c(X) = X^{(n-k)}m(X) + [X^{(n-k)}m(X) \bmod g(X)]$ ")
print("=" * 70)

def systematic_encode(m_poly, g, n, k):
    """체계적 인코딩 수행"""
    r = n - k
    shifted = X^r * m_poly
    _, p = shifted.quo_rem(g) # 나머지 = 패리티
    c_poly = shifted + p
    c_vec = vector(F, [c_poly[i] for i in range(n)])
    return c_poly, p, c_vec

test_messages = [
    ("(1,0,1,1)", 1 + X^2 + X^3),
    ("(1,1,0,0)", 1 + X),
    ("(0,0,0,1)", X^3),
    ("(0,1,1,0)", X + X^2),
    ("(1,1,1,1)", 1 + X + X^2 + X^3),
]

for label, m_poly in test_messages:
    c_poly, p, c_vec = systematic_encode(m_poly, g, n, k)
    p_vec = vector(F, [p[i] for i in range(n - k)])
    m_vec = vector(F, [m_poly[i] for i in range(k)])
    print(f" m = {label}, m(X) = {m_poly}")
    print(f" 패리티 p(X) = {p}, p = {list(p_vec)}")
    print(f" 부호어 c = {list(c_vec)}")
    print(f" 구조 확인: [패리티|메시지] = {list(p_vec)}|{list(m_vec)}")
    # 유효성 검증
    remainder = c_poly % g

```

```

    print(f" c(X) mod g(X) = {remainder} ({'OK' if remainder == 0 else 'X'})")
    print()

# =====
# 6. 전체 체계적 부호어 테이블 (예제 8.4)
# =====
print("=" * 70)
print("6. (7,4) 전체 체계적 부호어 테이블 (16개)")
print("=" * 70)
print(f"{'메시지':<12} {'m(X)':<20} {'패리티':<10} {'부호어':<16} {'wH':>4}")
print("-" * 65)

V4 = VectorSpace(F, 4)
codeword_set = set()

for m_vec in V4:
    m_poly = sum(F(m_vec[i]) * X^i for i in range(k))
    c_poly, p, c_vec = systematic_encode(m_poly, g, n, k)
    p_vec = [p[i] for i in range(n - k)]
    wH = sum(int(c_vec[i]) for i in range(n))
    codeword_set.add(tuple(c_vec))
    print(f" {list(m_vec)!s:<12} {str(m_poly):<20} {str(p_vec):<10} {list(c_vec)!s:<16} {wH:>4}")

print(f"\n총 부호어 수: {len(codeword_set)} (기대값: {2^k})")
print()

# =====
# 7. 순환 이동 불변성 검증
# =====
print("=" * 70)
print("7. 순환 이동 불변성 검증")
print("=" * 70)

def cyclic_shift(v, n):
    """벡터 v를 오른쪽으로 1칸 순환 이동"""
    v = list(v)
    return tuple([v[-1]] + v[:-1])

cyclic_ok = True
for cw in codeword_set:
    shifted = cw
    for s in range(1, n):
        shifted = cyclic_shift(shifted, n)
        if shifted not in codeword_set:
            print(f" X {cw} → {s}칸 이동 → {shifted} : 부호어 아님!")
            cyclic_ok = False

if cyclic_ok:
    print(" OK 모든 부호어의 모든 순환 이동이 부호어 집합에 포함된다.")
    print(" → 순환 부호(Cyclic Code) 확인!")
print()

# =====
# 8. 신드롬 테이블 (예제 9.3)
# =====

```

```

# =====
print("=" * 70)
print("8. 단일 비트 오류 신드롬 테이블")
print("=" * 70)
print(f"{'오류 위치 i':<14} {'e(X)':<12} {'s(X) = e(X) mod g(X)':<24} {'신드롬 벡터'}")
print("-" * 65)

syndrome_table = {}
for i in range(n):
    e_poly = X^i
    s_poly = e_poly % g
    s_vec = tuple(F(s_poly[j]) for j in range(n - k))
    syndrome_table[s_vec] = i
    print(f" {i:<14} {str(e_poly):<12} {str(s_poly):<24} {s_vec}")

print(f"\n서로 다른 신드롬 수: {len(syndrome_table)} (기대값: {n})")
all_distinct = len(syndrome_table) == n
print(f"모든 단일 오류 정정 가능: {'OK' if all_distinct else 'X'}")
print()

# =====
# 9. 신드롬 기반 오류 정정 시뮬레이션 (예제 9.4)
# =====

print("=" * 70)
print("9. 신드롬 기반 오류 검출 및 정정 시뮬레이션")
print("=" * 70)

# 원본 메시지 인코딩
m_orig = 1 + X^2 + X^3 # m = (1,0,1,1)
c_poly_orig, _, c_vec_orig = systematic_encode(m_orig, g, n, k)
print(f"원본 메시지: m = (1,0,1,1)")
print(f"부호어: c = {list(c_vec_orig)}")
print()

# 각 위치에 단일 비트 오류 주입 후 정정
print("단일 비트 오류 주입 → 신드롬 계산 → 정정:")
print("-" * 65)

for err_pos in range(n):
    # 오류 주입
    e_poly = X^err_pos
    v_poly = c_poly_orig + e_poly
    v_vec = vector(F, [v_poly[i] for i in range(n)])

    # 신드롬 계산
    s_poly = v_poly % g
    s_vec = tuple(F(s_poly[j]) for j in range(n - k))

    # 오류 위치 추정
    if s_vec in syndrome_table:
        est_pos = syndrome_table[s_vec]
        corrected_poly = v_poly + X^est_pos
        corrected_vec = vector(F, [corrected_poly[i] for i in range(n)])
        success = (corrected_vec == c_vec_orig)

```

```

        print(f" 오류 위치={err_pos}, 수신={list(v_vec)}, "
              f"신드롬={s_vec}, 추정 위치={est_pos}, "
              f"정정={'OK' if success else 'X'}")
    else:
        print(f" 오류 위치={err_pos}, 신드롬={s_vec}, 테이블에 없음!")

print()

# =====
# 10. 2비트 오류 시 오정정 확인 (예제 9.5)
# =====
print("=" * 70)
print("10. 2비트 오류 시 오정정(Miscorrection) 확인")
print("=" * 70)

e_poly_2bit = 1 + X # 위치 0, 1에 동시 오류
v_poly_2bit = c_poly_orig + e_poly_2bit
v_vec_2bit = vector(F, [v_poly_2bit[i] for i in range(n)])
s_poly_2bit = v_poly_2bit % g
s_vec_2bit = tuple(F(s_poly_2bit[j]) for j in range(n - k))

print(f"원본 부호어: {list(c_vec_orig)}")
print(f"오류 패턴: e(X) = 1 + X (위치 0, 1)")
print(f"수신 벡터: {list(v_vec_2bit)}")
print(f"신드롬: {s_vec_2bit}")

if s_vec_2bit in syndrome_table:
    est_pos = syndrome_table[s_vec_2bit]
    corrected = v_poly_2bit + X^est_pos
    corrected_vec = vector(F, [corrected[i] for i in range(n)])
    is_correct = (corrected_vec == c_vec_orig)
    print(f"추정 오류 위치: {est_pos} (실제: 0과 1)")
    print(f"정정 결과: {list(corrected_vec)}")
    print(f"정정 성공 여부: {'OK' if is_correct else 'X 오정정(Miscorrection) 발생!'})")
print()

# =====
# 11. 패리티 검사 다항식을 이용한 부호어 유효성 검사
# =====
print("=" * 70)
print("11. h(X)를 이용한 부호어 유효성 검사: c(X)*h(X) ≡ 0 mod (X^7+1)")
print("=" * 70)

h = (X^7 + 1) // g
print(f"h(X) = {h}")
print()

# 모든 부호어에 대해 검사
all_valid = True
for cw in codeword_set:
    c_poly_check = sum(F(cw[i]) * X^i for i in range(n))
    product = (c_poly_check * h) % (X^n + 1)
    if product != 0:
        print(f" X c = {cw}: c(X)*h(X) mod (X^7+1) = {product}")

```

```

all_valid = False

if all_valid:
    print(f" OK 모든 {len(codeword_set)}개 부호어에 대해  $c(X) \cdot h(X) \equiv 0 \pmod{X^7+1}$  성립")
print()

# =====
# 12. 다른 생성 다항식으로 순환 부호 생성 (예제 3.1)
# =====
print("=" * 70)
print("12.  $X^7+1$ 의 다양한 인수로 만드는 순환 부호들")
print("=" * 70)

generators = [
    ("1+X", 1 + X),
    ("1+X+X^3", 1 + X + X^3),
    ("1+X^2+X^3", 1 + X^2 + X^3),
    ("(1+X)(1+X+X^3)", (1 + X) * (1 + X + X^3)),
    ("(1+X)(1+X^2+X^3)", (1 + X) * (1 + X^2 + X^3)),
    ("(1+X+X^3)(1+X^2+X^3)", (1 + X + X^3) * (1 + X^2 + X^3)),
]

print(f"{'g(X)':<30} {'deg':<6} {'(n,k)':<10} {'d_min':<8} {'부호어 수'}")
print("-" * 70)

for label, gen in generators:
    try:
        C_test = codes.CyclicCode(generator_pol=gen, length=7)
        d_min = C_test.minimum_distance()
        dim = C_test.dimension()
        print(f" {label:<30} {gen.degree():<6} (7,{dim}){'':<5} {d_min:<8} {2^dim}")
    except Exception as e:
        print(f" {label:<30} 오류: {e}")

print()

# =====
# 13. LFSR 시뮬레이션 — 체계적 인코딩
# =====
print("=" * 70)
print("13. LFSR 시뮬레이션:  $g(X) = 1+X+X^3$  체계적 인코딩")
print("=" * 70)

def lfsr_systematic_encode(message_bits, g_coeffs, n, k):
    """
    LFSR을 이용한 체계적 인코딩 시뮬레이션.
    message_bits: 길이 k의 메시지 비트 리스트 (m_0, m_1, ..., m_{k-1})
    g_coeffs: g(X)의 계수 리스트 [g_0, g_1, ..., g_{n-k}]
    높은 차수 비트부터 입력 (m_{k-1}, m_{k-2}, ..., m_0)
    """
    r = n - k # 레지스터 수
    reg = [F(0)] * r # 레지스터 초기화

    print(f" {'클럭':<6} {'입력':<8} {'피드백':<10} {'레지스터':}")

```

```

print(f" {'초기':<6} {'-':<8} {'-':<10} {reg}")

# 높은 차수부터 입력
for clk in range(k):
    input_bit = F(message_bits[k - 1 - clk])
    feedback = input_bit + reg[r - 1] # 입력 XOR 최상위 레지스터

    # 레지스터 시프트 + 피드백 적용
    new_reg = [F(0)] * r
    new_reg[0] = feedback * F(g_coeffs[0]) # g_0 계수
    for j in range(1, r):
        new_reg[j] = reg[j - 1] + feedback * F(g_coeffs[j])

    reg = new_reg
    print(f" {clk+1:<6} {int(input_bit):<8} {int(feedback):<10} {[int(x) for x in reg]}")

return [int(x) for x in reg]

# g(X) = 1 + X + X^3 → 계수 [1, 1, 0] (g_0=1, g_1=1, g_2=0; g_3=1은 최고차항)
g_coeffs = [1, 1, 0] # g_0, g_1, g_2 (g_3=1은 피드백 경로 자체)
message = [1, 0, 1, 1] # m = (1, 0, 1, 1)

print(f"메시지: {message}")
print(f"g(X) = 1 + X + X^3, 피드백 계수: g_0={g_coeffs[0]}, g_1={g_coeffs[1]}, g_2={g_coeffs[2]}")
print()

parity_lfsr = lfsr_systematic_encode(message, g_coeffs, n, k)
print(f"\nLFSR 결과 (레지스터 상태): {parity_lfsr}")

# 다항식 나눗셈으로 구한 패리티와 비교
m_poly_check = 1 + X^2 + X^3
_, p_check = (X^3 * m_poly_check).quo_rem(g)
p_check_vec = [int(p_check[i]) for i in range(3)]
print(f"다항식 나눗셈 패리티: {p_check_vec}")
print()

# =====
# 14. (7,3) 순환 부호 예제 (예제 5.3)
# =====

print("=" * 70)
print("14. (7, 3) 순환 부호: g(X) = 1 + X^2 + X^3 + X^4")
print("=" * 70)

g73 = 1 + X^2 + X^3 + X^4
C73 = codes.CyclicCode(generator_pol=g73, length=7)
print(f"부호 파라미터: [{C73.length()}, {C73.dimension()}, {C73.minimum_distance()}]")
print(f"생성 다항식: g(X) = {g73}")
print()

# 생성 행렬
G73 = Matrix(F, 3, 7)
for i in range(3):
    poly = (X^i * g73) % (X^7 + 1)
    for j in range(7):

```

```

        G73[i, j] = poly[j]

print("생성 행렬 G:")
print(G73)
print()

# 인코딩 예제: m = (1, 1, 0)
m73 = vector(F, [1, 1, 0])
c73 = m73 * G73
print(f"m = (1,1,0) → c = {list(c73)}")
print()

# =====
# 15. SageMath 내장 CyclicCode와의 비교 검증
# =====

print("=" * 70)
print("15. SageMath 내장 CyclicCode와 수동 계산 비교")
print("=" * 70)

C_builtin = codes.CyclicCode(generator_pol=g, length=7)
G_builtin = C_builtin.systematic_generator_matrix()
print("SageMath 체계적 생성 행렬:")
print(G_builtin)
print()

# 수동 체계적 인코딩 결과와 비교
match_count = 0
total = 2^k
for m_vec in VectorSpace(F, k):
    # SageMath 내장
    c_builtin = m_vec * G_builtin

    # 수동 체계적 인코딩
    m_poly = sum(F(m_vec[i]) * X^i for i in range(k))
    _, _, c_manual = systematic_encode(m_poly, g, n, k)

    if list(c_builtin) == list(c_manual):
        match_count += 1

print(f"일치하는 부호어 수: {match_count}/{total}")
print(f"수동 인코딩과 SageMath 내장 결과 {'완전 일치 OK' if match_count == total else '불일치 존재 X'}")
print()

# =====
# 16. 준순환 행렬 블록 구조 확인 (예제 16.1)
# =====

print("=" * 70)
print("16. 순환 행렬(Circulant Matrix) 블록 구조")
print("=" * 70)

def circulant_matrix(first_row, field=F):
    """첫 번째 행으로부터 순환 행렬 생성"""
    m = len(first_row)
    M = Matrix(field, m, m)

```

```

    for i in range(m):
        for j in range(m):
            M[i, j] = first_row[(j - i) % m]
    return M

first_row = [F(1), F(0), F(1), F(1)]
C_circ = circulant_matrix(first_row)
print(f"첫 번째 행: {first_row}")
print("순환 행렬:")
print(C_circ)
print()

# 각 행이 이전 행의 순환 이동인지 확인
circ_ok = True
for i in range(1, 4):
    prev_row = list(C_circ[i - 1])
    expected = [prev_row[-1]] + prev_row[:-1]
    actual = list(C_circ[i])
    if actual != expected:
        circ_ok = False
        print(f" X 행 {i}: 기대 {expected}, 실제 {actual}")

if circ_ok:
    print(" OK 모든 행이 이전 행의 오른쪽 순환 이동 → 순환 행렬 확인!")
print()

print("=" * 70)
print("모든 검증 완료!")
print("=" * 70)

```

참고문헌

- [1] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, no. 3, pp. 379-423, 1948.
- [2] R. W. Hamming, "Error Detecting and Error Correcting Codes," Bell System Technical Journal, vol. 29, no. 2, pp. 147-160, 1950.
- [3] D. E. Muller, "Application of Boolean Algebra to Switching Circuit Design and to Error Detection," IRE Transactions on Electronic Computers, vol. EC-3, no. 3, pp. 6-12, 1954.
- [4] I. S. Reed, "A Class of Multiple-Error-Correcting Codes and the Decoding Scheme," IRE Transactions on Information Theory, vol. 4, no. 4, pp. 38-49, 1954.
- [5] E. Prange, "Cyclic Error-Correcting Codes in Two Symbols," Air Force Cambridge Research Center, AFCRC-TN-57-103, 1957.
- [6] A. Hocquenghem, "Codes correcteurs d'erreurs," Chiffres, vol. 2, pp. 147-156, 1959.
- [7] R. C. Bose and D. K. Ray-Chaudhuri, "On a Class of Error Correcting Binary Group Codes," Information and Control, vol. 3, no. 1, pp. 68-79, 1960.
- [8] I. S. Reed and G. Solomon, "Polynomial Codes Over Certain Finite Fields," Journal of the Society for Industrial and Applied Mathematics, vol. 8, no. 2, pp. 300-304, 1960.
- [9] W. W. Peterson, Error-Correcting Codes. Cambridge, MA: MIT Press, 1961.

- [10] E. R. Berlekamp, Algebraic Coding Theory. New York: McGraw-Hill, 1968.
- [11] J. L. Massey, "Shift-Register Synthesis and BCH Decoding," IEEE Transactions on Information Theory, vol. IT-15, no. 1, pp. 122-127, 1969.
- [12] V. D. Goppa, "Codes on Algebraic Curves," Soviet Mathematics Doklady, vol. 24, pp. 170-172, 1981.
- [13] R. J. McEliece, "A Public-Key Cryptosystem Based on Algebraic Coding Theory," DSN Progress Report, vol. 42-44, pp. 114-116, Jet Propulsion Laboratory, 1978.
- [14] C. Berrou, A. Glavieux, and P. Thitimajshima, "Near Shannon Limit Error-Correcting Coding and Decoding: Turbo-Codes," Proceedings of ICC '93, pp. 1064-1070, 1993.
- [15] D. J. C. MacKay and R. M. Neal, "Near Shannon Limit Performance of Low Density Parity Check Codes," Electronics Letters, vol. 32, no. 18, pp. 1645-1646, 1996.